Keyword-based Community Search in Bipartite Spatial-Social Networks (Technical Report)

Kovan A. Bavi
Kent State University
Kent, Ohio, USA
University of Zakho
Zakho, Kurdistan Region, Iraq
kmali@kent.edu
kovan.m.ali@uoz.edu.krd

Xiang Lian
Kent State University
Kent, Ohio, USA
xlian@kent.edu

ABSTRACT

Several approaches have been recently proposed for community search in bipartite graphs. These methods have shown promising results in identifying communities in real-world bipartite networks, such as social and biological networks. Given a query user q, community search in bipartite graphs involves identifying a group of users containing q, with common characteristics or functions within a given bipartite graph. These problems are particularly challenging because bipartite graphs have two distinct sets of nodes, and community search algorithms must account for this structure. However, finding communities in keyword-based bipartite spatial-social networks has yet to be investigated enough. The spatial-social networks are naturally structured as bipartite graphs. Thus, this paper proposes a new community search problem in Bipartite spatial-social networks with a novel (ω, π)-*keyword-core*, named *Keyword-based Community Search in Bipartite Spatial-Social Networks* (*KCS-BSSN*). The *KCS-BSSN* returns a tightly-knit community, significant social influence, minimal travel distance, and includes a (ω, π)-*keyword-core*. To address the *KCS-BSSN* problem, we have developed pruning methods that effectively filter out irrelevant users and points of interest. To improve query-answering efficiency, we have also proposed an indexing technique named the bipartite-spatial-social index. Our pruning techniques, and indexing approach, have proven effective and efficient through experiments with real and artificial data sets.

PVLDB Reference Format:
Kovan A. Bavi and Xiang Lian. Keyword-based Community Search in Bipartite Spatial-Social Networks (Technical Report). PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX
PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/KBavi-Personal/KCS-BSSN_WITH_DYNAMIC.

1 INTRODUCTION

The community search problem has recently attracted significant attention due to its wide range of practical applications in areas such as marketing [28], recommendation systems [30, 31], and team formation [30, 39]. With the rapid growth of location-based social networks (LBSNs), an enormous volume of social and spatial data has become available. These networks provide a comprehensive representation of human interactions in physical space by capturing both social relationships and mobility behaviors.

Spatial-social networks inherently consist of two distinct but interrelated components: users and spatial locations. Users are connected to their checked-in points of interest (POIs), which are embedded within a road network. Consequently, such networks naturally form a bipartite structure, where one partition represents users and the other represents POIs, and edges exist only between these two sets. Bipartite graphs thus model the interactions between upper-level entities (users) and lower-level entities (POIs), reflecting users' spatial activities and preferences.

Recently, several studies have investigated community search in bipartite graphs under various bipartite-core models [22, 23, 32, 35, 39, 41]. While these works focus on identifying cohesive structures within bipartite networks, research on spatial-social networks often overlooks their inherent bipartite nature. In many existing approaches, social and spatial data are analyzed separately when performing community search, treating the social graph and the road network as independent components. Such separation fails to capture the complex interactions between users and points of interest. To effectively form a meaningful community in spatial-social networks, several interrelated factors must be considered simultaneously: (i) structural cohesiveness among users, (ii) significant social influence within the group, (iii) minimal travel distance to relevant POIs, and (iv) the inherent bipartite relationships between users and POIs. Ignoring any of these aspects may lead to communities that are structurally valid but practically ineffective.

Social networks play a crucial role in shaping users' opinions and decision-making processes on specific topics. Influence propagation within tightly connected communities can significantly affect behavioral adoption. Meanwhile, users' checked-in locations correspond to POIs described by keywords, representing shared interests and preferences. By jointly considering social influence and POI-related keyword information, it becomes possible to identify communities that are both socially cohesive and semantically aligned with specific interests. Analyzing these dimensions independently cannot adequately model the complex interplay of social relationships and spatial behaviors. Therefore, it is essential to design a unified framework that fully exploits the bipartite nature of spatial-social networks.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

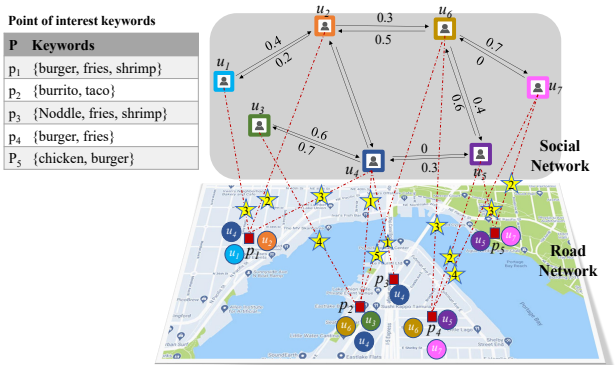


Figure 1: An example of bipartite spatial-social networks.

To address this gap, we introduce a new community search problem, namely *Keyword-based Community Search in Bipartite Spatial-Social Networks* (*KCS-BSSN*). The objective of *KCS-BSSN* is to retrieve a cohesive community that: (1) contains a query user q, (2) satisfies strong structural cohesiveness in the social network, (3) exhibits significant social influence, (4) minimizes travel distance between community members and relevant POIs, and (5) satisfies a (ω, π)-*keyword-core* constraint in the bipartite network. This integrated formulation enables the discovery of communities that are structurally strong, influential, spatially convenient, and semantically meaningful.

We further extend our *KCS-BSSN* framework to support Temporal Bipartite Spatial-Social Networks (*TBSSN*). By integrating temporal constraints into the core discovery process, we ensure that the identified communities are not only socially and spatially cohesive but also reflect the latest user behaviors (within the most recent sliding window). Our proposed approach can effectively filter out historical or obsolete interactions, providing results with high semantic relevance to current activity patterns.

Our proposed framework has a wide range of real-world applications, including urban planning, location-aware recommendation, and targeted marketing. Example 1 illustrates how our model can be applied for marketing purposes.

Example 1. ***(Online Advertising)*** *Figure 1 illustrates an example of a bipartite-spatial-social network. In this example, the social network G_s consists of vertices that represent users $u_1 \sim u_7$, and friendship among users shown by edges (e.g., $e(u_1, u_2)$). Each edge e has a weight representing the user's influence on a specific topic (e.g., traveling) between 0 and 1. On the other hand, in the road network G_r, the vertices represent intersection points, and the edges represent road segments that connect the vertices. In the social network G_s, each user could have multiple checked-in locations on G_r representing visited points of interest, where the points of interest $p_1 \sim p_5$ are located on the road network. The users from G_s visit the points of interest on G_r with a positive frequency number, illustrated inside stars on dotted edges. On the other hand, each p_i is associated with a list of keywords that describe that point of interest. The list of all keywords is presented in Figure 1.*

Consequently, a business wants to target its advertising toward a particular group of people who enjoy eating burgers and fries at restaurants. The chosen individuals should be geographically and socially close to each other. The group members should also be reasonably close to the restaurant. It is common for a group to choose a restaurant that some members have visited often in the past. The company may select a loyal customer as a reference user and can influence other's opinions. To ensure that the group is a good fit, each member should have prior experience visiting places that provide at least one shared service.

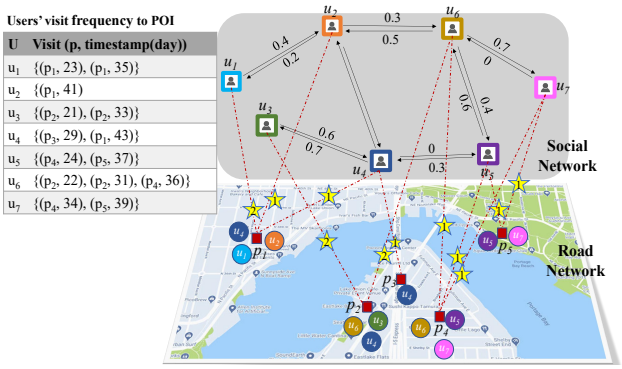


Figure 2: An example of a temporal bipartite spatial-social network with a visit updating frequency for the last 30 days, considering today as day #50.

Online advertising is usually not a one-time event, but a continuous process to affect the shopping attitudes and behaviors of users in the community. It is therefore important to continuously monitor potential customer groups (communities) in the bipartite spatial-social networks over time (namely temporal *BSSN*, or *TBSSN*, with dynamic updates, e.g., visiting frequencies of POIs by users).

We have the following motivation example on the continuous customer group monitoring for online advertising.

Example 2. ***(Continuous Customer Group Monitoring for Online Advertising)*** *Figure 2 illustrates a temporal bipartite-spatial-social network (TBSSN). The social network G_s consists of vertices representing users $u_1 \sim u_7$, where friendship ties are denoted by edges weighted by social influence. Within the road network G_r, POIs $p_1 \sim p_5$ are situated at specific intersections and associated with descriptive keywords. Each user in G_s maintains a set of checked-in locations on G_r; every visit from a user to a POI is associated with a specific timestamp (e.g., in days), as presented in the temporal visit vectors of Figure 2.*

Consider a business launching a limited-time burger promotion. To maximize conversion, the business seeks a target group that is geographically proximal, socially cohesive, and significantly influenced by a loyal reference user q. To ensure the advertisement reaches an active audience, the company imposes a recency constraint τ, that is, we are interested in those users who have frequently visited burger restaurants for the past 30 days. In this case, we can issue a KCS-TBSSN query, which dynamically considers these historical interactions, identifying a community that is structurally and spatially connected, and actively engaged with the company's products within a valid temporal window (i.e., a recent sliding window).

In Examples 1 and 2, we illustrate how to identify a community of users within a social network where members maintain close relationships, and a selected influential user can affect the opinions

of others. Such an influence is essential for effectively persuading the group to spread/propagate online advertising. Moreover, some members should have frequently visited the restaurant, thereby enhancing its reputation and credibility among their friends. Finally, the selected restaurant should be geographically convenient for all members, ensuring minimal travel distance and encouraging collective participation.

Most existing research on community search in bipartite graphs primarily focuses on identifying structural cores within bipartite networks. These studies emphasize structural properties but often overlook additional social and spatial dimensions. Conversely, prior work on spatial-social networks typically analyzes the social graph and the road network separately, without fully integrating their inherent bipartite relationships. In contrast, our work jointly considers multiple essential aspects. Specifically, we incorporate social cohesiveness by ensuring strong relationships among users, account for social influence by modeling how a selected user can affect others' opinions, and enforce spatial proximity by minimizing travel distance for all community members. Furthermore, we integrate the bipartite structure by prioritizing the reputation of Points of Interest (POIs), measured through users' visit frequencies. This unified approach enables the discovery of communities that are structurally cohesive, influence-aware, spatially compact, and semantically meaningful.

Challenges. Dealing with spatial-social networks can be challenging due to the vast amounts of information associated with these networks. In addition, the problem at hand involves many constraints that require considerable computation. Addressing these constraints separately can increase computational time. However, finding a solution that combines these constraints presents its own challenges.

Contributions. This paper makes the following contributions:

- **New Community Model:** we define (ω, π)-*keyword-core* a bipartite model that captures the complex interactions between social influence and road-network proximity (Section 2.4).
- **Novel Indexing:** We design a unified indexing tree and cost model that integrates social and road-network data to enable direct user-centric filtering within index nodes (Section 4).
- **Efficient Algorithm:** We propose a comprehensive *KCS-BSSN* algorithm featuring multi-stage pruning techniques to eliminate false-alarm users and irrelevant POIs (Sections 5 and 3).
- **Temporal Extension:** We extend *KCS-BSSN* to temporal networks, integrating time-aware constraints to ensure communities reflect the most recent user behaviors (Section 6).
- **Empirical Validation:** Extensive experiments on real and synthetic datasets demonstrate the superior efficiency and scalability of our solution over baseline approach (Section 7).

2 PROBLEM DEFINITION

In this section, we will formally define the data model for *bipartite spatial-social networks* (*BSSN*) and our *keyword-based community search* problem over *BSSN*.

2.1 Social Networks

In this subsection, we first give the data model for social networks below.

Definition 1. **(Social Network, G_s).** *A social network G_s is a graph in the triple form (V_s, E_s, θ_s), where V_s is a set of m user vertices u_1, u_2, ..., and u_m, E_s is a set of directed edges $e(u_j, u_k)$, each connecting two users u_j and u_k and associated with a weight $w(u_j, u_k)$, and θ_s is a mapping function $V_s \times V_s \rightarrow E_s$.*

In Definition 1, a social network can be considered as an influence graph, where the weight $w(u_j, u_k)$ of each edge $e(u_j, u_k) \in E_s$ is an influence of user u_j on user u_k, with respect to some topic or interest of the user (e.g., movie, sports, etc.).

The influence weight between users in G_s can be computed using the *text-based topic discovery algorithm* [4]. For simplicity, in this paper, we assume that each user influences other users only with respect to one single topic. We can easily extend our proposed solution to the scenario of multiple topics [2, 7], by keeping influence weight vectors for different topics, which we will leave it as our future work.

Influence Score Function. Assume that user vertices u and v can be directly or indirectly connected by a path, $u \rightsquigarrow v$, of length $(l-1)$ in social networks G_s, that is, $u = x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_l = v$, where x_i (for $1 \leq i \leq l$) is a vertex on the path. We define an *influence score function* (ISF) between any two user vertices u and v as follows.

Definition 2. **(Influence Score Function, *ISF* [7])** *Given a social network G_s, the influence score, $w(u \rightsquigarrow v)$, of user u on user v through a path $u \rightsquigarrow v$ with length $(l-1)$ is given by:*

$$w(u \rightsquigarrow v) = \prod_{i=1}^{l-1} w(x_i, x_{i+1}). \tag{1}$$

The influence score function (ISF), $ISF(u, v)$, is defined as the maximum influence score among all possible paths from u to v:

$$ISF(u, v) = \max_{\forall u \rightsquigarrow v} \{w(u \rightsquigarrow v)\}. \tag{2}$$

Note that, the influence score function (given by Eq. (2)) in Definition 2 is not symmetric (i.e., $ISF(u, v) \neq ISF(v, u)$). In other words, the influence of user u on user v could be different from that of user v on user u.

Social Cohesiveness. There are many existing techniques [10] to capture the cohesiveness of a community in social networks G_s, such as k-core [5, 11, 26], k-truss [8, 18, 34, 40], k-clique [1, 38], k-(edge connected component) [14, 37], etc.

In this paper, we consider the (k, d)-*truss* [19] as follows.

Definition 3. **((k, d)-*truss* [19]).** *Given a social network G_s, a query user $q \in V_s$, and two positive integers k (> 2) and d, a (k, d)-truss is a connected subgraph $g \subseteq G_s$, such that: (1) each edge $e \in E_s(g)$ is contained in at least $(k-2)$ triangles, and; (2) for any user $u \in V_s(g)$, $dist_s(q, u) \leq d$ holds, where $dist_s(q, u)$ is the shortest path distance between q and u on the social network G_s.*

Intuitively, the (k, d)-*truss* in Definition 3 returns a community with high connectivity of graph structures (i.e., with $\geq (k-2)$ triangles) and with friend relationships close to the query user q (i.e., within d hops away from q).

2.2 Spatial Road Networks

Next, we define a spatial road network, G_r, as follows.

Definition 4. **(Spatial Road Network, G_r).** *A spatial road network, G_r, is a planar graph, represented by a triple (V_r, E_r, θ_r), where V_r is a set of n vertices (i.e., intersection points) $r_1, r_2, ..., and\ r_n$, E_r is a set of edges $e(r_j, r_k)$ (each connecting two intersection points r_j and r_k), and θ_r is a mapping function $V_r \times V_r \rightarrow E_r$.*

In Definition 4, the road network G_r is a graph, with intersection points as vertices and road line segments as edges. Each vertex $r_i \in V_r$ has its 2D location, $r_i.\ell$, with longitude and latitude, ($r_i.x$, $r_i.y$), in Euclidean space.

In the road network G_r, there are *points of interest* (POIs) on road segments (edges), defined as follows.

Definition 5. **(Points of Interest, V_p).** *Given a spatial road network G_r, we have a set, V_p, of points of interest (POIs) on edges in E_r, where each POI $p \in V_p$ is associated with its 2D location $p.\ell$ $(= (p.x, p.y))$ and a list, $p.K$, of its descriptive keywords.*

Examples of POIs in Definition 5 include restaurants, hotels, cinemas, airports, etc.

2.3 Bipartite Spatial-Social Networks

Users in social networks G_s can have one or multiple checked-in locations (i.e., POIs), $u.L$, on spatial road networks G_r. Here, checked-in locations can be obtained via GPS or WiFi location services from social networks (e.g., Twitter or Yelp).

Essentially, users over social networks G_s and POI locations on spatial road networks G_r can form a checked-in bipartite graph, defined as follows.

Definition 6. **(Bipartite Spatial-Social Network, G_b).** *Given users in V_s on social networks G_s and POIs in V_p on spatial road networks G_r, a bipartite spatial-social network (BSSN), G_b, is a bipartite graph, in the form of a quadruple $(V_s, V_p, E_b, \mathcal{F}_b)$, where the edge set E_b contains edges from user vertices $u \in V_s$ to POIs $p \in V_p$, and function $\mathcal{F}_b$ is a mapping: $E_b \rightarrow R^+$ that assigns each edge $e(u, p)$ with a positive, real-valued weight $f_{u,p}$.*

In Definition 6, we model the checked-in relationships between users and POIs as a bipartite graph (i.e., *BSSN*), where each mapping edge $e(u, p)$ in E_b from a user $u \in V_s$ to a POI $p \in V_p$ is associated with a weight $f_{u,p}$. In practice, the edge weight $f_{u,p}$ can be the frequency that user u visits the POI p.

2.4 Keyword-Based Community Search Over Bipartite Spatial-Social Networks

Keyword-Weight-Constrained Community, (ω, π)-*keyword-core*. In this subsection, we will define the problem of *keyword-based community search over bipartite spatial-social networks* (*KCS-BSSN*). Before that, we first introduce the concept of (ω, π)-*keyword-core* over bipartite spatial-social networks G_b.

Definition 7. **((ω, π)-*keyword-core*).** *Given a bipartite spatial-social network G_b, a query keyword set Q, and parameters ω and π, a (ω, π)-keyword-core, $B = (V'_s, V'_p, E'_b, \mathcal{F}'_b)$, is a connected, maximal bipartite subgraph of G_b, such that:*

- *each POI $p \in V'_p$ contains at least one query keyword (i.e., $p.K \cap Q \neq \emptyset$);*
- *each user $u \in V'_s$ has the summed visiting frequency $f_{sum}(u, V'_p) = \sum_{p \in V'_p} f_{u,p} \geq \omega$, and;*
- *each POI $p \in V'_p$ has the average visiting frequency $f_{avg}(V'_s, p) = \frac{\sum_{u \in V'_s} f_{u,p}}{|\{u \in V'_s | f_{u,p} \neq 0\}|} \geq \pi$, where $|\{u \in V'_s | f_{u,p} \neq 0\}| \neq 0$.*

In Definition 7, we define a bipartite community (i.e., (ω, π)-*keyword-core*), B, satisfying the constraints of keywords and aggregated edge weights. In particular, each POI $p \in V'_p$ must contain at least one query keyword in Q (i.e., $p.K \cap Q \neq \emptyset$). Moreover, each user $u \in V'_s$ should have his/her summed frequency of visits no less than ω (i.e. $f_{sum}(u, V'_p) \geq \omega$), which indicates the preference of user u to POIs in V'_s (with the specified query keywords). Similarly, each POI $p \in V'_p$ must have the average visiting frequency $f_{avg}(V'_s, p)$ higher than or equal to π, which implies the popularity of the POI p.

Definition 8. **(The Average Spatial Distance Function).** *Given a social-network user, $u \in V_s$ and $p \in V_p$, we calculate the $avg_dist_r(u, p)$ as the average shortest path distance between all the user u's checked-in locations $u.L$ and the $p.\ell$*

$$avg_dist_r(u, p) = \frac{\sum_{i=1}^{|u.L|} dist_r(u.L_i, p.\ell)}{|u.L|}, \tag{3}$$

where $|u.L|$ is the total number of points of interest p visited by the user u, and $dist_r(., .)$ is the shortest path distance between two locations in the road network G_r.

Definition 8 gives the average distance $avg_dist_r(u, p)$ between user u and a POI p, which intuitively captures the spatial closeness between user u and POI p (visited by other users) in G_r. The small average distance indicates the possibility of recommending POI p to user u within the same bipartite community in real applications such as online marketing and advertising.

Keyword-Based Community Search Over Bipartite Spatial-Social Networks. With constraints of keyword, (aggregated) edge weights, and average user-POI distances, we are now ready to define the keyword-aware community search over bipartite spatial-social networks (*KCS-BSSN*) as follows:

Definition 9. **(Keyword-Based Community Search Over Bipartite Spatial-Social Networks, *KCS-BSSN*).** *Given a bipartite spatial-social network G_b, a keyword query set Q, parameters k, d, ω, and π, a spatial distance threshold σ, an influence score threshold θ, and a query user q, a keyword-based community search over bipartite spatial-social networks (KCS-BSSN) returns a maximal bipartite subgraph (community), $B = (V'_s, V'_p, E'_b, \mathcal{F}'_b)$, of G_b such that:*

- *$q \in V'_s$;*
- *V'_s is a (k, d)-truss;*
- *for any user $u \in V'_s$, we have the influence score $ISF(q \rightsquigarrow u) \geq \theta$;*
- *for any user $u \in V'_s$ and POI $p \in V'_p$, it holds that $avg_dist_r(u, p) \leq \sigma$;*
- *the bipartite subgraph B is a (ω, π)-keyword-core, and;*
- *any subgraph $B' \supset B$ is not a KCS-BSSN community.*

Intuitively, in Definition 9, the *KCS-BSSN* problem returns a maximal group, $B = (V'_s, V'_p, E'_b, \mathcal{F}'_b)$, of users (including query user q) in social networks G_s and their interested (or frequently visited) POIs $p \in V_p$ on road networks, where any user u in V'_s have impact influence by q (i.e., $\geq \theta$) in G_s, any user u in V'_s has close average

Table 1: Notations and Descriptions

Symbol	Description
G_s	social networks
G_r	spatial road networks
G_b	bipartite spatial-social networks
V_p	a set of points of interest (POIs)
$w(u, v)$	the weight on edge $e(u, v)$ of G_s
ISF	influence score function
V_s	a set of users in G_b
$f_{u,p}$	the frequency of a user u visiting the POI p in G_b
Q	a query keyword set
q	a query user
σ	a spatial distance threshold
θ	an influence score threshold
ω	a user summed visiting frequency threshold
π	a POI average visiting frequency threshold
τ	a timestamp threshold

road-network distance to POIs p, and POIs p in V'_p contain some query keyword(s) in Q.

2.5 Challenges

The problem *KCS-BSSN* (as given in Definition 9) is rather challenging to tackle, in terms of efficiency. One straightforward method is as follows: we first enumerate all possible users in social networks (including the query user q) based on the (k, d)-*truss* properties (as given in Definition 3) and the *influence score* (as given in Definition 2), then prune all POIs $p \in V_p$ that do not contain any keywords in the query keyword set Q, check the constraints of (ω, π)-*keyword-core* (as given in Definition 7), and finally examine the distance between users $u \in V'_s$ and POIs $p \in V'_p$. This straightforward method is, however, not efficient, due to a large number of possible communities (with an exponential number of possible user-and-POI combinations). Therefore, in the sequel, we will design effective pruning techniques to filter out as many false alarms of users/POIs as possible to reduce the problem search space, and develop an indexing mechanism to enable our proposed efficient *KCS-BSSN* query processing algorithm.

3 PRUNING TECHNIQUES

In this section, we provide pruning techniques to rule out as many false alarms of users in social networks and POIs on road networks as possible.

3.1 Keyword-Based Pruning

In this subsection, we present a *keyword-based pruning* method, which filters out those users from the social network G_s based on their previous visits to POIs V_p (without any keywords in the query keyword set Q).

Lemma 1. **(Keyword-Based Pruning).** *Given a bipartite graph G_b, a query keyword set Q, and a set, V_p, of POIs that user u visited in G_b, user u can be safely pruned, if $p.K \cap Q = \emptyset$ holds for all POIs $p \in V_p$, where $p.K$ is a keyword set associated with POI p.*

Proof. The proof is provided in the Appendix A. □

3.2 ω-Based Pruning

As given in Definition 7, any user u who is in a community $C \in G_s$ must satisfy the condition that $f_{sum}(u, V'_p) \geq \omega$. Thus, our ω-*based pruning* method aims to filter out any user u with low $f_{sum}(u, V'_p)$ (i.e., $< \omega$). However, directly computing $f_{sum}(u, V'_p)$ requires an online summation of $f_{u,p}$ for all POIs $p \in V'_p$, which is rather costly. To accelerate the process, we will alternatively obtain an upper bound, $ub_f_{sum}(u, V'_p)$, of $f_{sum}(u, V'_p)$ offline, and online prune a user u, if it holds that $ub_f_{sum}(u, V'_p) < \omega$.

We have the following ω-based pruning lemma.

Lemma 2. **(ω-based Pruning).** *Given a user u, a POI set V_p, and a threshold ω, any user u can be safely pruned, if $ub_f_{sum}(u, V'_p) < \omega$ holds, where $ub_f_{sum}(u, V'_p)$ is an upper bound of $f_{sum}(u, V'_p)$.*

Proof. The proof is provided in the Appendix A. □

Discussions on How to Compute $ub_f_{sum}(u, V'_p)$: In Lemma 2, we need to compute the upper bound $ub_f_{sum}(u, V'_p)$ of $f_{sum}(u, V'_p)$. Note that, V'_p in the community may not include all POIs that user u has visited before. Therefore, we can sum up $f_{u,p}$ for all POIs in V_p ($\supseteq V'_p$) that user u has visited, and obtain this upper bound $ub_f_{sum}(u, V'_p) = f_{sum}(u, V_p) = \sum_{p \in V_p} f_{u,p} \geq \sum_{p \in V'_p} f_{u,p} = f_{sum}(u, V'_p)$.

3.3 π-Based Pruning

Based on Definition 7, any POI p in (ω, π)-*keyword-core* must have the average visiting frequency $f_{avg}(V'_s, p) \geq \pi$, where $V'_s (\subseteq V_s)$. Then, we can filter out any POI p if $f_{avg}(V'_s, p) < \pi$. However, our π-*based pruning* method aims to filter out any user u who does not visited any p with high $f_{avg}(V'_s, p)$ (i.e., $\geq \pi$). Thus, to avoid calculating the average of $f_{u,p}$ online for all $p \in V'_p$ visited by user u, we calculated offline the upper bound $ub_f_{avg}(u)$ (i.e., $ub_f_{avg}(u) = max_{\forall p \in V_p} f_{u,p}$). The calculated $ub_f_{avg}(u)$ can help prune u for an online query if it holds that $ub_f_{avg}(u) < \pi$.

In Lemma 3, we can safely prune false alarm users based on π-based pruning.

Lemma 3. **(π-Based Pruning).** *Given a user u, a POI set V_p, and thresholds π, the u can be safely pruned, if $ub_f_{avg}(u) < \pi$.*

Proof. The proof is provided in the Appendix A. □

Discussions on How to Compute $ub_f_{avg}(u)$: In order to prune user u in Lemma 3, we need to compute $ub_f_{avg}(u)$. Therefore, we obtain the upper $ub_f_{avg}(u) = max_{\forall p \in V_p} f_{u,p}$, where the upper bound $ub_f_{avg}(u)$ holds for all POIs p in V'_p ($\subseteq V_p$) that have been visited by the user u.

3.4 Influence-Based Pruning

In order to ensure that the influence score between any two users u, v in $S \subseteq G_s$ satisfies the constraint $ISF(u \rightsquigarrow v) \geq \theta$ (as given in Definition 9), we need to compute the influence score between all pairs in S. However, it is possible for any pair to have multiple paths between them, making the calculation of ISF for online queries very complex. Therefore, for each $u \in V_s$, we computed the upper bound In-influence (Out-influence) $ub_w_{in}(u)$ ($ub_w_{out}(u)$) offline, respectively. Later, we obtain online the upper bound influence score $ub_ISF(.,.)$ between any two $u, v \in V_s$ based on previously calculated $ub_w_{in}(.)$ and $ub_w_{out}(.)$.

Then, we pruned any user v if $ub_ISF(u, v) < \theta$ as stated in the following Influence-based pruning lemma.

Lemma 4. **(Influence-Based Pruning).** *Given a social network G_s, and a threshold θ, for any two users $u, v \in V(G_s)$, we can safely prune v, if $ub_ISF(u,v) < \theta$ holds.*

Proof. Derived from the Definition 9. □

Discussions on How to Compute $ub_ISF(u,v)$: Chen et al. [7] proposed techniques to efficiently compute the upper bound influence among users, one of which is the *Neighborhood-Based Estimation*. For each user $u \in V_s$, let in-neighbors, $\mathcal{N}_{in}(u) = \{v | \exists e(v,u) \in E_s\}$, and out-neighbors, $\mathcal{N}_{out}(u) = \{v | \exists e(u,v) \in E_s\}$. Then, we compute the In-influence (Out-influence) upper bound as $ub_w_{in}(u) = \max_{v \in \mathcal{N}_{in}(u)}\{w(v,u)\}$ ($ub_w_{out}(u) = \max_{v \in \mathcal{N}_{out}(u)}\{w(u,v)\}$), respectively.

Then, the upper bound influence score between any two vertices u, v can be computed as follows:

$$ub_ISF(u,v) = \begin{cases} \max\{ub_w_{out}(u), ub_w_{in}(v)\}, & \text{if } e(u,v) \in E_s; \\ ub_w_{out}(u) \cdot ub_w_{in}(v), & \text{if } e(u,v) \notin E_s. \end{cases} \quad (4)$$

Clearly, in Eq. (4), we consider only one direct neighbor of vertex u to compute the upper bound influence score. However, [7] and [2] confirmed that the proposed techniques are practical and efficient.

3.5 Structural Cohesiveness Pruning

The (k,d)-*truss* (as given in Definition 3) has a cohesiveness constraint that indicates that $\forall e \in \{E(H) | H \subseteq G_s\}$ must be contained in at least $(k-2)$ triangles denoted $sup(e)$. However, despite only having to compute $sup(e)$ for $e \in E(H)$, it is still inefficient for online queries. Thus, we computed $sup(e)$ offline for all $e \in E_s$. Then, based on $sub(e)$, we calculate the upper bound $ub_sup(.)$ for each user $u \in G_s$.

The pruning users from G_s based on structural cohesiveness pruning is provided in the following lemma.

Lemma 5. **(Structural Cohesiveness Pruning).** *Given a social network G_s, and an integer k, for any $u \in G_s$, we can safely prune u if $ub_sup(u) < k-2$.*

Proof. The proof is provided in the Appendix A. □

Discussions on How to Compute $ub_sup(u)$: For each edge $e \in E_s$, we computed $sup(e)$. Then, for each user u, we computed the maximum $sub(e)$ from its in-neighbors, $\mathcal{N}_{in}(u)$, and out-neighbors, $\mathcal{N}_{out}(u)$, as follows:

$$\begin{aligned} & ub_sup(u) \quad (5) \\ = \; & \max\left\{ \max_{\forall v \in \mathcal{N}_{out}(u)} sup(e(u,v)), \max_{\forall v \in \mathcal{N}_{in}(u)} sup(e(v,u)) \right\}. \end{aligned}$$

3.6 Social-Distance-Based Pruning

In addition to the cohesiveness constraint, the Definition 3 requires the distance constraint among users in $C \subseteq G_s$, which is the number of hops between any user $u \in C$ and the query user q. For a query user q and any user $u \in C$, if $dist_s(u,q) > d$, then we can safely prune u. However, computing the distance in G_s, online, between q and all users in G_s could be time-consuming, especially in large social networks. Thus, we precompute (offline) the distance between u and a set of pivot points in G_s, denoted $\mathbb{P}_s$. Then, for an online query, we easily obtain the lower bound distance $lb_dist_s(.)$ between two users utilizing the triangle inequality.

For a query user q and any user $u \in C$, if $lb_dist_s(u,q) > d$, then we can safely prune u. Then, we formally provide the following lemma.

Lemma 6. **(Social-Distance-Based Pruning).** *Given a social network G_s, a query user q, and a social distance threshold d, for any $u \in V_s$, we can safely prune u if it holds that $lb_dist_s(u,q) > d$.*

Proof. Derived from the Definition 3. □

Discussions on How to Compute $lb_dist_s(.)$: We precompute (offline) the distance between all users $u \in G_s$ and all $spv_i \in \mathbb{P}_s$, where $\mathbb{P}_s = \{spv_1, \ldots, spv_{\mathfrak{a}}\}$. Then, for an online query, we use the triangle inequality to calculate the lower bound distance $lb_dist_s(.)$ between two users.

For any $u \in V_s$ and q, the $dist_s(u,q) \geq |dist_s(u, spv_i) - dist_s(q, spv_i)|$. Then, we can compute the lower bound distance between u and q, as follows:

$$lb_dist_s(u,q) = \max_{\forall spv_i \in \mathbb{P}_s} \{|dist_s(u, spv_i) - dist_s(q, spv_i)|\}. \quad (6)$$

The Social Network Pivots $\mathbb{P}_s$ Selection. To enhance our pruning, we must identify a set of pivot users within G_s based on the lower bound distance between any user in G_s and a query user q. We have created a cost model $\mathbb{P}_s_Cost$, as follows:

$$\begin{aligned} & \mathbb{P}_s_Cost \quad (7) \\ = \; & \sum_{\forall u \in G_s} \sum_{\forall v \in G_s} \max_{\forall spv_i \in \mathbb{P}_s} \{|dist_s(u, spv_i) - dist_s(v, spv_i)|\}. \end{aligned}$$

When dealing with online queries, we typically compare and prune any user $u \in C$ against the query user q. However, we calculate our cost model against all users in the social network (i.e. any user query) and choose the set with the minimum cost.

3.7 Spatial-Distance-Based Pruning

Calculating the average distance (as given in Eq. (3)) for online queries can be very costly. For a user u with $u.L$, we have to compute $avg_dist_r(.)$ between u and $\forall p \in V'_p$, which requires $O(|u.L| \cdot |V'_p|)$ complexity. In order to answer online queries faster, we precompute (offline) the distance between $\forall p \in V_p$ to a set of pivot points in G_r, denoted $\mathbb{P}_r$. Then, we utilize the triangle inequality for an online query to calculate the lower bound average distance $lb_avg_dist_r(.)$ more efficiently.

For a user u and $\forall p \in \{q$'s checked-in locations $|p.K \cap Q \neq \emptyset\}$, if $lb_avg_dist_r(u,p) > \sigma$, then we can safely prune u. For this purpose, we formally provide the following lemma.

Lemma 7. **(Spatial-Distance-Based Pruning).** *Given a social network G_s, a spatial road network G_r, a query user q, a keyword query set Q, and a spatial distance threshold σ, we can safely prune any user $u \in V_s$, if $\forall p \in \{q$'s checked-in locations $|p.K \cap Q \neq \emptyset\}$ the $lb_avg_dist_r(u,p) > \sigma$ holds.*

Proof. Derived from the Definition 9. □

Discussions on How to Compute $lb_avg_dist_r(.)$: We precompute (offline) the distance between all POIs $p \in V_p$ and all $rpv_i \in \mathbb{P}_r$ in the road network G_r, where $\mathbb{P}_r = \{rpv_1, \ldots, rpv_{\mathfrak{b}}\}$. Then, for an

online query, we use the triangle inequality to calculate the lower bound average distance $lb_avg_dist_r(.)$ between a user u and any POI p that the query user q visited and satisfies the constraint $p.K \cap Q \neq \emptyset$.

For any $u \in V_s$, and any $p \in \{q$'s checked-in locations $|p.K \cap Q \neq \emptyset\}$, the $dist_r(u.L_i, p.\ell) \geq |dist_r(u.L_i, rpv_i) - dist_r(p.\ell, rpv_i)|$. Then, we can compute the average distance lower bound, as follows:

$$lb_avg_dist_r(u,p) = \frac{\sum_{j=1}^{|u.L|} \max_{\forall rpv_i \in \mathbb{P}_r} \{dist_r(u.L_j, rpv_i) - dist_r(p, rpv_i)|\}}{|u.L|}. \quad (8)$$

The Road Network Pivots $\mathbb{P}_r$ Selection. To choose a set of pivot points on the road network, we must first calculate a cost model that considers the user's checked-in location and the location of the POIs on the road network. To maximize the pruning effectiveness of the $lb_avg_dist_r(u,p)$, our chosen set of $\mathbb{P}_r$ must have the maximum average distance between each $u \in G_s$ and $p \in V_p$ on one side and the $rpv_i \in \mathbb{P}_r$ on the other. We then calculate the cost model to select $\mathbb{P}_r$, as follows:

$$\mathbb{P}_r_Cost = \sum_{\forall u \in G_s} \sum_{\forall p \in V_p} \frac{\sum_{j=1}^{|u.L|} \max_{\forall rpv_i \in \mathbb{P}_r} \{dist_r(u.L_j, rpv_i) - dist_r(p, rpv_i)|\}}{|u.L|} \quad (9)$$

The lowest value of $\mathbb{P}_r_Cost$ will enhance the pruning power of techniques in Spatial-Distance-Based Pruning.

4 INDEXING MECHANISM

In the following subsections, we describe the indexing mechanism for *KCS-BSSN* queries and the corresponding pruning strategies applied to index nodes.

4.1 Index Structure

To enable efficient *KCS-BSSN* query processing, we construct a tree index $\mathcal{I}$ over the bipartite spatial-social network. The social network is first partitioned into subgraphs, each stored in a leaf node. These leaf nodes are then recursively grouped into intermediate nodes until a single root node is formed. The index tree consists of two types of nodes:

Leaf Node. Each leaf node N contains a set of users from the social network. For each user $u \in N$, we maintain:

- a keyword set $u.K = key_1, \ldots, key_{|K|}$ with aggregate statistics f_{sum} and f_{max}. Let P_u denote the set of POIs visited by u. The keyword aggregates are computed as:

$$key_j.f_{sum} = \sum_{\forall p \in P_u, key_j \in p.K} f_{u,p} \quad (10)$$

$$key_j.f_{max} = \max_{\forall p \in P_u, key_j \in p.K} f_{u,p} \quad (11)$$

- an upper bound edge support $ub_sup(u)$, where $ub_sup(u)$= $\max\{sup(e)\}$.
- an In-influence upper bound $ub_w_{in}(u)$ and an Out-influence upper bound $ub_w_{out}(u)$.
- a vector of social distance to each social pivot $spv_i \in \mathbb{P}_s$, $\{dist_s(u, spv_1), dist_s(u, spv_2), \ldots, dist_s(u, spv_b)\}$, where $\mathbb{P}_s$ is a set of pivot points in social network.

Non-Leaf Node. Each non-leaf node N contains a set of child nodes N_i. For each such node, we maintain aggregated upper-bound information:

- a keyword set $N.K = key_1, \ldots, key_{|K|}$ with upper bounds:

$$key_j.ub_f_{sum} = max_{\forall u \in N}\{key_j.f_{sum}\} \quad (12)$$

$$key_j.ub_f_{max} = max_{\forall u \in N}\{key_j.f_{max}\} \quad (13)$$

- an upper bound edge support:

$$ub_sup(N) = max_{\forall u \in N}\{ub_sup(u)\} \quad (14)$$

- an upper bound in-influence:

$$ub_w_{in}(N) = \{max_{\forall u \in N} ub_w_{in}(u)\} \quad (15)$$

- a vector of minimum and maximum social distances between node N and each pivot $spv_i \in \mathbb{P}s$:

$$mindist_s(N, spv_i) = \min_{\forall u \in N}\{dist_s(u, spv_i)\} \quad (16)$$

$$maxdist_s(N, spv_i) = \max_{\forall u \in N}\{dist_s(u, spv_i)\} \quad (17)$$

4.2 Index-Level Pruning

Our pruning techniques provided in Section 3 are for pruning users and points of interest. However, applying those techniques directly to large-scale databases is very costly. Since all users in $\mathcal{I}$ are bounded by MBRs, we can use this property by pruning the entire MBR. In the following subsection, we will provide pruning techniques at the index level to prune a set of false alarm users.

4.2.1 Keyword-based Pruning for Index Nodes. As discussed in Section 4.1, every node N is associated with a set $N.K$, then we can prune any N if $N.K \cap Q = \emptyset$. In the following lemma, we formally provide Keyword-based Pruning for index nodes.

LEMMA 8. **(Keyword-based Pruning for Index Nodes).** *Given an index node N and a keyword query set Q, the N can be safely pruned, if $N.K \cap Q = \emptyset$.*

In Lemma 8, if $N.K \cap Q = \emptyset$ indicates that for any $u \in N$, the POI visited by u do not have any key in Q. Then, node N can be safely pruned.

4.2.2 ω-based Pruning for Index Nodes. Since for any $key_j \in N.K$, the $key_j.ub_f_{sum}$ is the upper bound f_{sum} for key_j (as given in Eq. (12)). Then, we can directly prune any node N if it holds $\max_{\forall key_j \in N.K | key_j \in Q} key_j.ub_f_{sum} < \omega$.

The pruning nodes of $\mathcal{I}$ based on ω-based pruning are provided in the following lemma.

LEMMA 9. **(ω-based Pruning for Index $\mathcal{I}$ Nodes).** *Given an index node N, a keyword query set Q, and thresholds ω, the N can be safely pruned, if $\max_{\forall key_j \in N.K | key_j \in Q} key_j.ub_f_{sum} < \omega$.*

In Lemma 9, if $\max_{\forall key_j \in N.K | key_j \in Q} key_j.ub_f_{sum} < \omega$ indicates that for any $u \in N$ the $\max_{\forall key_j \in u.K | key_j \in Q} key_j.ub_f_{sum} < \omega$. Then, node N can be safely pruned.

4.2.3 π-based Pruning for Index Nodes. As given in Eq. (13), for each $key_j \in N.K$, the $key_j.ub_f_{max}$ is the upper bound of f_{max} for all $u \in N$. Then, for this pruning, we can filter out any $N \in \mathcal{I}$ if $\max_{\forall key_j \in N.K | key_j \in Q} key_j.ub_f_{max} < \pi$. Then, we formally provide the following lemma.

LEMMA 10. **(π-based Pruning for Index $\mathcal{I}$ Nodes).** *Given an index node N, a keyword query set Q, and thresholds π, node N can be safely pruned, if* $\max_{\forall key_j \in N.K | key_j \in Q} key_j.ub_f_{max} < \pi$.

In Lemma 10, if $\max_{\forall key_j \in N.K | key_j \in Q} key_j.ub_f_{max} < \pi$, then for all $u \in N$ the $\max_{\forall key_j \in u.K | key_j \in Q} key_j.ub_f_{max} < \pi$. In this case, we can safely prune the node N.

4.2.4 Influence-based Pruning for Index Nodes. Since we calculated for each $N \in \mathcal{I}$ the $ub_w_{in}(N)$ (as given in Eqs. (15)), we can easily compute the $ISF(q \rightsquigarrow N)$ between a query user q and node N. Then, for any node N if $ISF(q \rightsquigarrow N) < \theta$, we can safely prune N. We estimate the upper bound influence score between a query user q and node N as follows:

$$ub_ISF(q, N) = ub_w_{out}(q) \cdot ub_w_{in}(N). \tag{18}$$

LEMMA 11. **(Influence-based Pruning for Index Nodes).** *Given an index node N and a threshold θ, for any q, we can prune safely N if $ub_ISF(q, N) < \theta$.*

In Lemma 11, if $ub_ISF(q, N) < \theta$ confirms that for any $u \in N$ the $ub_ISF(q, u) < \theta$, then node N can be safely pruned.

4.2.5 Structural Cohesiveness Pruning for Index Nodes. For any node N, we can safely remove N if for all its child users u hold $ub_sup(u) < k - 2$. Since $ub_sup(N)$ is the upper bound edge support for all $u \in N$, as stated in Eq. (14). Then, we formally provide the following lemma.

LEMMA 12. **(Structural Cohesiveness Pruning for Index Nodes).** *Given an index node N and an integer k, for any $N \in \mathcal{I}$, we can safely prune N if $ub_sup(N) < k - 2$.*

In Lemma 12, the upper bound $ub_sup(N) < k - 2$ indicates that for any $u \in N$ the $ub_sup(u) \leq ub_sup(N)$. Then node N can be safely filtered out.

4.2.6 Social-distance-based Pruning for Index Nodes. In order to take advantage of the distance constraint between users in $C \subseteq G_s$ (as given in Definition 3), we can filter out a node $N \in \mathcal{I}$ if N does not satisfy the distance constraint. Specifically, for a given query user q if $dist_s(q, N) > d$, we can safely prune node N. Since for each $N \in \mathcal{I}$, we store the minimum and maximum social distance between N and every $spv_i \in \mathbb{P}_s$. Then, we can easily compute the $lb_dist_s(q, N)$ utilizing the triangle inequality.

For a query user q and any node $N \in \mathcal{I}$, if $lb_dist_s(q, N) > d$, then we can safely prune N. Then, we formally provide the following lemma.

LEMMA 13. **(Social-distance-based Pruning for Index Nodes).** *Given an index $\mathcal{I}$, a query user q, and a social distance threshold d. For any $N \in \mathcal{I}$, we can safely prune N if $lb_dist_s(q, N) > d$.*

Discussions on How to Compute $lb_dist_s(.)$: As discussed in Section 4.1, we calculate the minimum and maximum social distance between N and all $spv_i \in \mathbb{P}_s$. Then, for a given q and any $N \in \mathcal{I}$, the $dist_s(q, N) \geq |dist_s(q, spv_i) - dist_s(N, spv_i)|$. We can compute the lower bound distance between q and N as follows:

$$\begin{aligned} & lb_dist_s(q, N) \\ = \quad & \min \begin{cases} \max_{\forall spv_i \in \mathbb{P}_s} \{|dist_s(q, spv_i) - mindist_s(N, spv_i)|\} \\ \max_{\forall spv_i \in \mathbb{P}_s} \{|dist_s(q, spv_i) - maxdist_s(N, spv_i)|\} \\ 0, \quad if\, q \in N \end{cases}, \end{aligned} \tag{19}$$

where $mindist_s(.)$ and $maxdist_s(.)$ are given in Eqs. (16) and (17), respectively.

4.3 The Index Construction

To optimize query performance, our index construction ensures that each leaf node encapsulates a subgraph that strongly reflects prevalent *KCS-BSSN* community characteristics. Such localization increases the likelihood that a query result resides within a small number of leaf nodes, thereby reducing traversal overhead in the index tree $\mathcal{I}$. Algorithm 1 refines the set of pivot index users $\mathbb{P}_{index}$ using the cost model described in Section 5.1. We first compute the cost of an initial pivot set $\mathbb{P}_{index}$. During each iteration, a pivot user $piv_i \in \mathbb{P}_{index}$ is swapped with a candidate user $u \in G_s$ ($u \neq piv_i$), and the new cost is evaluated. After the refinement process, the pivot set with the minimum cost is selected. Algorithm 2 then partitions the social network G_s using the optimized pivot set $\mathbb{P}_{index}$. Each user v is assigned to the pivot $piv_i \in \mathbb{P}_{index}$ that maximizes a quality function defined in Section 5.1. The resulting subgraphs form the leaf nodes of the index tree. These leaf nodes are recursively grouped into intermediate nodes, where a cost model determines the most suitable parent-child relationships. For each intermediate node, a subset $\mathbb{P}'_{index} \subset \mathbb{P}_{index}$ is selected following a similar pivot selection strategy.

More specifically, **Algorithm 2: Partition_Social_Network** takes G_s and $\mathbb{P}_{index}$ as input. For each user u, the algorithm evaluates all pivots and assigns u to the pivot that yields the highest quality score (lines 1–8). This produces a set of disjoint subgraphs, which serve as the foundation for index construction. We first initialize the required local variables (line 2) and then evaluate each $piv_i \in \mathbb{P}_{index}$ to identify the pivot that provides the highest quality score for user u (lines 3–7). In line 8, user u is assigned to the corresponding subgraph. The algorithm ultimately outputs the resulting set of subgraphs.

In **Algorithm 1: Pivot_Index_Refinement**, the inputs include the social network G_s, the spatial network G_r, and a parameter *threshold-iter* specifying the maximum number of refinement iterations. Initially, $\mathfrak{s}$ pivot users are randomly selected (line 1), and corresponding subgraphs $S_1, \ldots, S_{\mathfrak{s}}$ are generated using Algorithm 2. For up to *threshold-iter* iterations, one pivot piv_i is randomly replaced with a candidate *temp-piv* (lines 4–5). The network is repartitioned using the updated pivot set (line 7), and the new cost is computed using Eq. (20). If the new configuration achieves a lower cost, it replaces the current pivot set (lines 8–9). After convergence or completion of iterations, the final pivot set $\mathbb{P}_{index}$ and its corresponding subgraphs are returned.

5 *KCS-BSSN* QUERY ANSWERING

In this section, we present **Algorithm 3, the** *KCS-BSSN* **Query Answering Algorithm**, designed for the efficient retrieval of

Algorithm 1: Pivot_Index_Refinement

Input: a social network G_s, a spatial network G_r, $threshold\text{-}iter$
Output: a set of pivot index $\mathbb{P}_{index}$

$\mathbb{P}_{index}$= Randomly select $\mathfrak{s}$ of $\mathbb{P}_{index}$
$Partition_Social_Network(G_s, \mathbb{P}_{index})$
while $iter < threshold\text{-}iter$ **do**
  Randomly select a new pivot user $temp\text{-}piv \in G_s$
  Randomly select a pivot user $piv_i \in \mathbb{P}_{index}$
  $temp\text{-}\mathbb{P}_{index} = \mathbb{P}_{index} - \{piv_i\} + \{temp\text{-}piv\}$
  $Partition_Social_Network(G_s, temp\text{-}\mathbb{P}_{index})$
  if $temp\text{-}\mathbb{P}_{index}_cost < \mathbb{P}_{index}_cost$ **then**
    $\mathbb{P}_{index} = temp\text{-}\mathbb{P}_{index}$
$Partition_Social_Network(G_s, \mathbb{P}_{index})$
return $\{\mathbb{P}_{index}\}$

Algorithm 2: Partition_Social_Network

Input: a social network G_s, a set of pivot index $\mathbb{P}_{index}$
Output: a set of subgraphs $S_1, \ldots, S_{\mathfrak{s}}$

for ***each*** *user* $u \in G_s$ **do**
  i=1; $best_quality = 0$; $j = i$
  while $i \leq \mathfrak{s}$ **do**
    if $quality(u, piv_i) > best_quality$ **then**
      $best_quality = quality(u, piv_i)$
      $j = i$
    $i = i + 1$;
  $S_j = S_j + \{u\}$
return $\{S_1, \ldots, S_{\mathfrak{s}}\}$

community results. The algorithm adopts a "filter-and-refine" framework, partitioned into a Pruning Phase and a Refinement Phase. It takes as input the social network G_s, spatial network G_r, weighted bipartite network G_b, keyword query set Q, structural thresholds k and d, keyword/influence thresholds ω, π, and θ, a spatial distance threshold σ, and the query user q.

Pruning Phase: The algorithm begins by initializing POI_q, the set of q's checked-in locations that contain at least one keyword from Q (line 1). To facilitate an efficient search, two priority queues are initialized: a max-heap $\mathcal{H}$ for index tree traversal and a min-heap $\mathcal{H}_{cand}$ for candidate management. $\mathcal{H}$ stores entries (N, *heap-key*), where N is an index node and *heap-key* represents the upper bound edge support, $ub_sup(N)$ (line 2). $\mathcal{H}_{cand}$ stores potential candidate users $u \in G_s$ prioritized by the lower bound average road distance, $lb_avg_dist_r(u, p)$, (line 3). The traversal begins by pushing the root of the index tree $\mathcal{I}$ into $\mathcal{H}$ (line 4). While $\mathcal{H}$ is not empty, the top node N is extracted (line 6). If its *heap-key* falls below $k - 2$, the search terminates early as no remaining nodes can satisfy the structural cohesiveness requirements (line 7). If N is a leaf node, we iterate through each user $u \in N$ and apply a suite of user-level pruning filters, including keyword, ω, π, influence, structural, social-distance, and spatial-distance pruning. Users who survive these filters are inserted into $\mathcal{H}_{cand}$ (lines 9-12). If N is a non-leaf node, we apply index-level pruning to its children. Any child node N_i that cannot be pruned is inserted into $\mathcal{H}$ with its corresponding upper bound support (lines 15-17).

Algorithm 3: *KCS-BSSN*_Query_Answer

Input: a social network G_s, a spatial network G_r, a weighted bipartite network G_b, a keyword query set Q, query thresholds k, d, ω, and π, a spatial distance threshold σ, an influence score threshold θ, and a query user q.
Output: a community C, satisfying *KCS-BSSN* (as given in Definition 9)

1 set $POI_q = \{q$'s checked-in locations $|p.K \cap Q \neq \emptyset\}$
2 initialize a max-heap $\mathcal{H}$ accepting entries in the form (N, $heap\text{-}key$)
3 initialize a min-heap $\mathcal{H}_{cand}$ accepting entries in the form (u, $heap\text{-}key_{cand}$)
4 insert entry $(root(\mathcal{I}), 0)$ into heap $\mathcal{H}$
5 **while** $\mathcal{H}$ *is not empty* **do**
6 (N, *heap-key*) = de-heap $\mathcal{H}$
7 **if** *heap-key* $< k - 2$, **then**
8 break and terminate.
9 **if** N *is a leaf node* **then**
10 **for** ***each*** *user* $u \in N$ **do**
11 **if** u *cannot be pruned by Lemma 1, 2, 3 4, 5, 6, and 7, w.r.t* q **then**
12 insert $\left(u, min_{\forall p \in POI_q}(lb_avg_dist_r(u, p))\right)$ into the heap $\mathcal{H}_{cand}$
13 **else**
14 // N *is a non-leaf node*
15 **for** ***each*** *entry* $N_i \in N$ **do**
16 **if** N_i *cannot be pruned by Lemma 8, 9, 10, 11, 12, and 13* **then**
17 insert $(N_i, ub_sup(N_i))$ into the heap $\mathcal{H}$
18 C= Refinement ($\mathcal{H}_{cand}$) //*refinement phase*
19 **return** C

Refinement Phase: The refinement phase (line 18) processes the candidate set stored in $\mathcal{H}_{cand}$. We first perform a threshold check: any candidate with a $lb_avg_dist_r$ exceeding σ is discarded, along with all remaining entries in the min-heap. For the remaining candidates, we verify the connected subgraphs against the full suite of *KCS-BSSN* requirements as defined in Definition 9. Finally, the algorithm returns the exact communities that satisfy all constraints.

5.1 The Index Pivots Selection for Leaf Nodes

To partition the social network into manageable subgraphs, we select $\mathfrak{s}$ specific users to serve as index pivots ($\mathbb{P}_{index}$). We utilize a cost model to identify the most suitable pivots by evaluating the social network across three dimensions: *Bipartite Structure*, *Social Structure*, and *Spatial Structure*. Each dimension is represented by a specific scoring function, which is integrated into the total cost calculation.

Total Cost. The cost associated with selecting index pivots $\mathbb{P}_{index}$ for all subgraphs $S \in G_s$ is defined as follows:

$$
\begin{aligned}
& \mathbb{P}_{index}_cost \qquad (20)\\
= \; & W_{bs} \cdot (1 - \sum_{\forall S \in G_s} \sum_{\forall u \in S} \sum_{\forall v \in S} bs_score(u, v)) \\
& + W_{rs} \cdot \sum_{\forall S \in G_s} \sum_{\forall u \in S} \sum_{\forall v \in S} rs_score(u, v) \\
& + W_{ss} \cdot (1 - \sum_{\forall S \in G_s} \sum_{\forall u \in S} \sum_{\forall v \in S} ss_score(u, v))
\end{aligned}
$$

where bs_score, rs_score, and ss_score represent the bipartite, road-network (spatial), and social structure scores, respectively (detailed in the Appendix B). The coefficients W_{bs}, W_{ss}, and W_{rs} are user-defined weights.

The Quality of Selecting Index Pivots $\mathbb{P}_{index}$. To assign a user u to a specific pivot $piv_i \in \mathbb{P}_{index}$, we evaluate their assignment quality based on the three structural scores:

$$
\begin{aligned}
& quality(u, piv_i) \qquad (21)\\
= \; & W_{bs} \cdot bs_score(u, piv_i) + W_{ss} \cdot ss_score(u, piv_i) \\
& + (1 - (W_{rs} \cdot rs_score(u, piv_i))
\end{aligned}
$$

This quality metric ensures that each user u is mapped to the ideal pivot piv_i that maximizes the structural and spatial coherence of the resulting leaf node.

5.2 The Index Pivots Selection for Non-Leaf Nodes

Constructing the intermediate levels of the index tree requires a distinct set of pivots, $\mathbb{P}'_{index}$. For non-leaf nodes, the cost model prioritizes the *Bipartite* and *Social* structures to maintain hierarchical cohesiveness.

Tree Cost. The index tree cost is calculated by evaluating the structural affinity of intermediate nodes as follows:

$$
\begin{aligned}
& Tree_cost \qquad (22)\\
= \; & W_{bs} \cdot (1 - \sum_{\forall N \in Nodes} \sum_{\forall piv'_i \in \mathbb{P}'_{index}} bs_score_node(N, piv'_i)) \\
& + W_{ss} \cdot (1 - \sum_{\forall N \in Nodes} \sum_{\forall piv'_i \in \mathbb{P}'_{index}} ss_score_node(N, piv'_i))
\end{aligned}
$$

The node-level scores bs_score_node and ss_score_node are defined in the Appendix C.

The Quality of Assigning Nodes to Parent Nodes. To construct the hierarchy, each child node N_i is assigned to a parent node N by evaluating the quality of the node relative to the non-leaf pivots:

$$
\begin{aligned}
& quality_node(N, piv'_i) \qquad (23)\\
= \; & W_{bs} \cdot bs_score_node(N, piv'_i) + W_{ss} \cdot ss_score_node(N, piv'_i)
\end{aligned}
$$

The index tree $\mathcal{I}$ is built bottom-up; for each $piv'_i \in \mathbb{P}'_{index}$, a new node N is generated, and child nodes N_i are assigned to the parent node that yields the highest quality. This process iterates until the root node is established, resulting in a balanced, structurally-aware index.

6 KEYWORD-BASED COMMUNITY SEARCH OVER TEMPORAL BIPARTITE SPATIAL-SOCIAL NETWORK *KCS-TBSSN*

Real-world (bipartite) graphs usually evolve dynamically over time, such as with edge weight updates. In this work, we also extend the original static *BSSN* graph to the data model of *temporal bipartite spatial-social network* (*TBSSN*), by considering edge weights $f_{u,p}$ dynamically change over a temporal dimension. Specifically, in the original model (Definition 6), edge weight $f_{u,p}$ represents a static/fixed frequency of historical visits. In our temporal *TBSSN* model, we re-define the relationships between users and POIs to account for the time-varying nature of user-POI interactions.

6.1 Dynamic Edge Weight Updates Under the Sliding Window Model

In the *TBSSN* graph, each user $u \in V_s$ is associated with a 2D temporal visit vector, $\mathcal{T}_{u,p}$, where each element $(p, t') \in \mathcal{T}_{u,p}$ represents a visit from user u to POI p at a specific timestamp t'. We consider a *sliding window* of size τ for obtaining the edge weight $f_{u,p}$. In particular, given the current timestamp t, the frequency $f_{u,p}$ is calculated as the count of all visits occurring for the most recent τ timestamps, that is,

$$f_{u,p}(t) = |\{(p, t') \in \mathcal{T}_{u,p} \mid t - \tau + 1 \leq t' \leq t\}|.$$

At a new timestamp $(t+1)$, new visits $(p, t+1) \in \mathcal{T}_{u,p}$ are added to $f_{u,p}(t)$, and expired visits $(p, t - \tau + 1) \in \mathcal{T}_{u,p}$ will be removed from $f_{u,p}(t)$, which result in an updated edge weight $f_{u,p}(t+1)$ for the new sliding window between timestamps $(t - \tau + 2)$ and $(t+1)$.

6.2 Updates of Temporal *TBSSN* Graph

To optimize query processing, we introduce a *temporal pruning* strategy. This mechanism identifies and discards "stale" interactions before the core computation begins. As established in Definition 7, if $f_{u,p}$ is updated and equal to zero, the edge will be removed from the graph. This approach offers two primary advantages:

(1) **Dynamic Validity:** By evicting invalid/expired visits at the start of the query, the resulting edge weights $f_{u,p}$ always represent the latest state of the network relative to the users' temporal constraints.
(2) **Space Reduction:** By removing the frequency of visits before timestamp $(t - \tau + 1)$ (t is the current timestamp), the edge weights may drop to zero, and thus the edges no longer exist. This can reduce the space cost of the graph storage. For example, we would not need the visiting records ten years ago, as it may not reflect the current popularity of POIs by users.

6.3 Dynamic Updates of Pre-Computed Data and Index for *KCS-TBSSN*

This subsection details the mechanisms for dynamically updating temporal *TBSSN* graph with low computational overhead. To manage updates on user-POI interactions, we address two main scenarios: including the frequency of new visits in a user's activity vector and removing the frequency of the expired visits that outside

the sliding window (i.e., before timestamp $(t - \tau + 1)$). To maintain high throughput, we employ a batch updating strategy. That is, rather than processing each check-in individually, we aggregate multiple updates in a batch to synchronize/update the *TBSSN* graph data. These updates can propagate changes to two critical components: the validity of community answers and the structural integrity of the indexing tree $\mathcal{I}$ (provided in Section 4).

6.3.1 Incremental Maintenance of Communities. For a user $u \in V_s$ and a set of active communities, we verify the validity of each community C as follows:

Insertion. For a new visit from user u to POI p:

- **Case 1 ($u \in C$):** If u is already a member of community C, the update is recorded, and we simply increment u's visit frequency. Structural cohesiveness remains unchanged.
- **Case 2 ($u \notin C$):** If $u \notin C$, we evaluate the potential inclusion of u into the community $C \cup \{u\}$. We apply the multi-stage pruning (i.e., the refinement phase of Algorithm 3). If $C \cup \{u\}$ satisfies all constraints, u is integrated; otherwise, the update is discarded for that specific C.

Deletion (Expiration). When an old visit by user u at timestamp t' expires (i.e., $t' < (t - \tau + 1)$, for current timestamp t):

- **Case 1 ($u \in C$):** If $u \in C$, the reduction in frequency may violate the (ω, π)-*keyword-core* constraints. We trigger the refinement process for C. If C fails any constraint (e.g., $f_{sum}(.) < \omega$), it is removed from the community answer set of our *KCS-BSSN* problem.
- **Case 2 ($u \notin C$):** The expiration has no impact on the community's validity and is ignored.

6.3.2 Indexing Level Maintenance. The index tree $\mathcal{I}$ is built on the relationships between users and POIs. The batch updates modify frequencies and associations, the cost model metrics—specifically the bipartite score (bs_score), road-network score (rs_score), and social score (ss_score)—may change over time, where bs_score, rs_score, and ss_score are given in Eqs. (24), (25) and (26), respectively.

To support efficient for query processing over temporal *TBSSN* graph (with dynamic updates), we perform the maintenance of index tree $\mathcal{I}$ as follows:

Boundary Constraint Validation. Following each batch update, we perform a validation of the index structure for both leaf nodes and non-leaf nodes, as detailed in Section 4.1. This process ensures the structural integrity of the hierarchy by verifying that all upper and lower bounds constraints are strictly maintained for every user and index nodes.

Incremental Pivot Re-evaluation. We monitor the cumulative change in $quality(u, piv_i)$ (given by Eq. (21)). For each user u in a leaf node, if a visit pattern changes such that another pivot $piv_j \in \mathbb{P}_{index}$ offers a significantly higher quality score, u is migrated to the subgraph of piv_j. A stability margin is used to avoid unnecessary updates.

Structural Synchronization. Leaf node updates propagate upward. For any node N, if the difference in $quality_node(N, piv'_i)$ (Eq. (24)) exceeds a specific margin relative to another piv'_j, we remap N to the most qualified upper-level node. This ensures pruning properties remain robust against temporal drift.

Algorithm 4: Dynamic_Maintenance

Input: Batch updates ΔU, OP (insertions/deletions), threshold τ, offline-data, a set of active communities $\mathbb{C}$, an Index tree $\mathcal{I}$, and a stability margin δ

Output: Updated (offline-data, $\mathbb{C}'$, $\mathcal{I}'$)

1 **for *each*** *update* $(u, p, t') \in \Delta U$ **do**
2 **if** $OP = Insertion$ **then**
3   **for *each*** *checkin-location* p **do**
4    $f_{u,p} \leftarrow f_{u,p} + |\{(p, t') \in \mathcal{T}_{u,p}|$
5 **else if** $OP = Deletion$ **then**
6   **for *each*** *checkin-location* p **do**
7    $f_{u,p} \leftarrow |\{(p, t') \in \mathcal{T}_{u,p} \mid t' \geq (t - \tau + 1)\}|$
8 Recompute user u offline-data
9 $\mathbb{C}' = Communities_Maintenance(\mathbb{C}, \Delta U, OP)$
10 $\mathcal{I}' = Indexing_Tree_Maintenance(\mathcal{I}, \Delta U$, offline-data, δ)
11 **return** (offline-data, $\mathbb{C}'$, $\mathcal{I}'$)

Algorithm 5: Communities_Maintenance

Input: A set of active communities $\mathbb{C}$, batch updates ΔU, OP (insertion/deletion)

Output: Updated set of valid communities $\mathbb{C}'$

for *each* $(u, p, t') \in \Delta U$ **do**
 for *each* $C \in \mathbb{C}$ **do**
  if $OP = Insertion$ **then**
   if $u \notin C$ **then**
    if $Refinement(C \cup \{u\})$ *satisfies constraints* **then**
     $C \leftarrow C \cup \{u\}$
  else if $OP = Deletion$ **then**
   if $u \in C$ **then**
    $C \leftarrow Refinement(C)$;   `// Trigger pruning`
    if $C = \emptyset$ **then**
     Remove C from $\mathbb{C}$;
return $\mathbb{C}'$

6.4 Incremental Community Maintenance Upon Dynamic Updates

In this subsection, we detail the algorithmic framework for managing temporal network changes. The process is governed by three primary procedures: **Algorithm 4** for high-level data synchronization, **Algorithm 5** for the community validity, and **Algorithm 6** for structural index integrity. In **Algorithm 4: Dynamic_Maintenance**, the procedure coordinates the high-level synchronization of the temporal network state. The process begins by iterating through each update (u, p, t') in the batch ΔU (line 1). Depending on the operation type, it either increments the visit frequency $f_{u,p}$ for new insertions (lines 2–4) or recalculates it based on the temporal threshold τ for deletion/expiration (lines 5–7). After updating these frequencies, the user's offline-data are re-computed to reflect current visit patterns (line 8). The algorithm then triggers sub-routines for community maintenance (line 9) and indexing tree maintenance (line 10) before returning the synchronized dataset (line 11).

Algorithm 6: Indexing_Tree_Maintenance

Input: Index tree $\mathcal{I}$, batch updates ΔU, updated offline-data, stability margin δ
Output: Synchronized Index tree $\mathcal{I}'$

1 **for** ***each*** *leaf-node* $L \in \mathcal{I}$ **do**
2 **for** ***each*** *user* $u \in (L \cap \Delta U)$ **do**
3 $piv_{curr} \leftarrow u.pivot$
4 $piv_{best} \leftarrow \text{argmax}_{piv_j \in \mathbb{P}_{index}} quality(u, piv_j)$
5 **if** $quality(u, piv_{best}) > quality(u, piv_{curr}) + \delta$ **then**
6 Migrate u to leaf-node of piv_{best}
7 Update boundary constraints (Upper/Lower bounds)
8 **for** ***each*** *non-leaf node* $N \in \mathcal{I}$ *(bottom-up)* **do**
9 Update boundary constraints (Upper/Lower bounds)
10 $piv'_{curr} \leftarrow N.parent_pivot$
11 $piv'_{best} \leftarrow \text{argmax}_{piv'_j} quality_node(N, piv'_j)$
12 **if** $quality_node(N, piv'_{best}) > quality_node(N, piv'_{curr}) + \delta$ **then**
13 Remap N to piv'_{best} and propagate updates upward;
14 **return** $\mathcal{I}'$

Furthermore, in **Algorithm 5: Communities_Maintenance**, the procedure ensures that active communities $\mathbb{C}$ remain valid according to all constraints. For each update in the batch (line 1), it iterates through existing communities (line 2) and checks the operation type. For an Insertion (line 3), if a user u is not already a member, it evaluates the community's validity including u through the $Refinement$ process (lines 5-6). For a Deletion (line 7), if the user u is a member, the $Refinement$ process is triggered to reevaluate the community based on the reduced frequency (line 9). If the community becomes empty during this process, it is removed from the active set (line 11). Finally, the set of survived communities $\mathbb{C}'$ is returned (line 12).

Finally, in **Algorithm 6: Indexing_Tree_Maintenance**, the hierarchical index I is updated to handle temporal drift and maintain search efficiency. The algorithm first performs Leaf-Level Migration: for every leaf node L and affected user u (lines 1–2), it compares the $quality$ of the current pivot versus all alternative pivots in $\mathbb{P}_{index}$ (lines 3-4). In lines 5-6, if an alternative pivot offers an improvement greater than the current leaf node with the stability margin δ, the user is migrated. Boundary constraints for the leaf node are then updated (line 7). Finally, the algorithm performs a Bottom-Up synchronization (line 8). For each non-leaf node N, it updates boundary constraints and evaluates the $quality_node$ score (line 9). If a more suitable parent pivot is found (exceeding δ), the node is remapped, and updates are propagated upward to ensure the pruning properties of the index remain robust against temporal changes (lines 10–13). Then, in line 14, we return a synchronized Index tree $\mathcal{I}'$.

7 EXPERIMENTAL EVALUATION

7.1 Experimental Settings

We evaluate the efficiency of our proposed algorithm using both real-world and synthetic datasets.

Real-World Datasets. We use three widely adopted social networks: Epinions [25] ($Epin$), Twitter [20] ($Twit$), and DBLP [36] ($DBLP$). Each edge $e(u, v)$ is assigned a weight in $(0, 1]$ following a $Gaussian$ distribution to model the influence of u on v. To construct spatial-social networks, users are mapped to 2D coordinates on the California road network [21] using a $Uniform$ distribution. Each user is associated with $[1, 10]$ check-in locations, with visit frequencies in $[1, 10]$, both uniformly distributed. Dataset statistics are summarized in Table 2.

Table 2: Statistics of Real-World Graph Datasets

Name	Nodes	Edges
Epinions social network [25]	75879	508837
Twitter [20]	81306	1768149
DBLP collaboration network [36]	317080	1049866

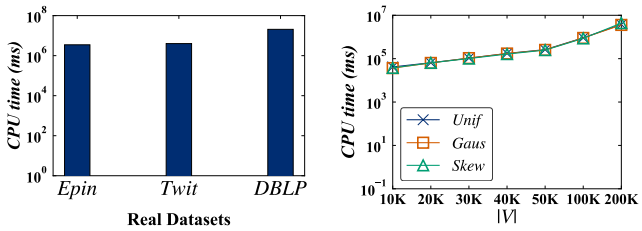
Table 3: Parameter Settings

Parameter	Values
the number of users $\|V_s\|$ in G_s	10K, 20K, **30K**, 40K, 50K, 100K, 200K
the number of triangles k in G_s	2, **3**, 4, 5, 6
the social distance threshold d	1, 2, **3**, 4, 5
the size of query keyword set $\|Q\|$	3, 5, **7**, 9, 11
the users visiting frequency threshold ω	0.2, **0.4**, 0.6, 0.7, 0.9
the POIs visited frequency threshold π	0.2, **0.4**, 0.6, 0.7, 0.9
the influence score threshold θ	0.2, **0.4**, 0.6, 0.7, 0.9
the spatial distance threshold σ	1, 2, 3, 4, **5**, 6

Synthetic Datasets. We generate an artificial social network G_s with varying numbers of users, where each user connects to $[8, 40]$ other users. For each edge $e(u, v)$, we assign a weight in $(0, 1]$ following a $Gaussian$ distribution to represent social influence. A road network G_r is constructed by generating $20K$ intersection points in a 2D space and connecting them using the Gabriel Graph Algorithm [13]. POIs are placed uniformly along road edges and assigned $[1, 8]$ keywords selected from a dictionary of up to 50 terms. We utilize three distributions—$Uniform$, $Gaussian$, and $Skew$ (Zipf skewness = 0.8)—to generate three synthetic datasets, denoted as $Unif$, $Gaus$, and $Skew$, respectively. The social and road networks are integrated into a bipartite spatial-social network, where each user is assigned $[1, 10]$ check-ins with visit frequencies in $[1, 10]$, both following a $Uniform$ distribution.

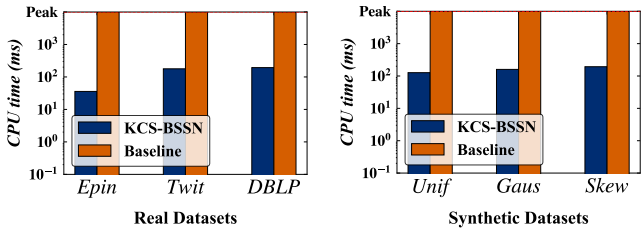
Temporal Dynamic Updates Evaluation. We evaluate the efficiency of our maintenance mechanisms by measuring the computational overhead across batch sizes $|\Delta U| \in \{10, 15, 25, 50, 100, 200\}$ on both the real-world $Twit$ and the synthetic $Unif\ TBSSN$ graphs. The users in $|\Delta U|$ were selected randomly following a $Uniform$ distribution. The evaluation is categorized into four key areas: first, $Data_B$ assesses the time required to synchronize check-in frequencies and recompute offline spatial-social metrics. To validate active community answers, we compare $Comm_1$, which represents the average CPU time to update community answers by testing affected users individually, against $Comm_B$, the average time for processing the entire update batch simultaneously to minimize $Refinement$ triggers and redundant checks. Finally, $Tree_B$ evaluates the cost of updating hierarchical boundary constraints and performing user and node migrations within the index $\mathcal{I}$. Each experiment is conducted for both Insertion and Deletion operations to simulate the sliding window model for affected users in the batch.

Evaluation Methodology. All datasets are indexed using our proposed indexing tree, and experiments are conducted on the constructed bipartite spatial-social networks. To the best of our knowledge, this is the first experimental study of the *KCS-BSSN* query. We evaluate efficiency by comparing the runtime of our



(a) real-world graphs (b) synthetic graphs

Figure 3: The *KCS-BSSN* offline time vs. real/synthetic graph datasets.



(a) real-world graphs (b) synthetic graphs

Figure 4: The *KCS-BSSN* performance vs. real/synthetic graph datasets.

approach with a baseline method. In the baseline, five subgraphs within social distance d from the query user q are sampled, and the total CPU cost is estimated by multiplying their average runtime by the total number of possible subgraphs within distance d in G_s. In each experiment, we vary one parameter while fixing the others to their default values (highlighted in bold in Table 3). The thresholds ω and π are normalized to $(0, 1]$, where 0.1 and 1 denote the minimum and maximum possible values, respectively. All experiments were conducted on a machine with an Intel Core i7 2.8GHz CPU and 16GB RAM.

7.2 *KCS-BSSN* Performance Evaluation

The *KCS-BSSN* Offline Processing Time vs. Real/Synthetic Datasets. Figure 3 reports the offline processing time of *KCS-BSSN*, including data preparation and indexing tree construction, on both real and synthetic datasets. For real datasets (Figure 3(a)), networks are processed at their original sizes (Table 2). For synthetic datasets (Figure 3(b)), we vary the social network size $|V|$ to evaluate scalability, while keeping other parameters at their default values. Results show a clear correlation between $|V|$ and offline time, confirming that dataset size significantly impacts preprocessing cost.

The *KCS-BSSN* Performance vs. Real/Synthetic Datasets. Figure 4 compares our approach with the baseline under default parameter settings. For real datasets (Figure 4(a)), original network sizes are used; for synthetic datasets (Figure 4(b)), $|V|$ is fixed at its default value. In all cases, *KCS-BSSN* consistently outperforms the baseline. The baseline frequently reaches peak execution times due to the exponential number of subgraphs it explores, whereas our approach remains stable and scalable across different data environments.

Performance vs. Query Keyword Set Size $|Q|$. Figure 5(a) evaluates performance for $|Q| \in 3, 5, 7, 9, 11$. Runtime increases moderately as $|Q|$ grows, since more keywords expand the number of matching POIs and candidate users. Nevertheless, the overall variation in computation time remains small, demonstrating robustness to keyword expansion.

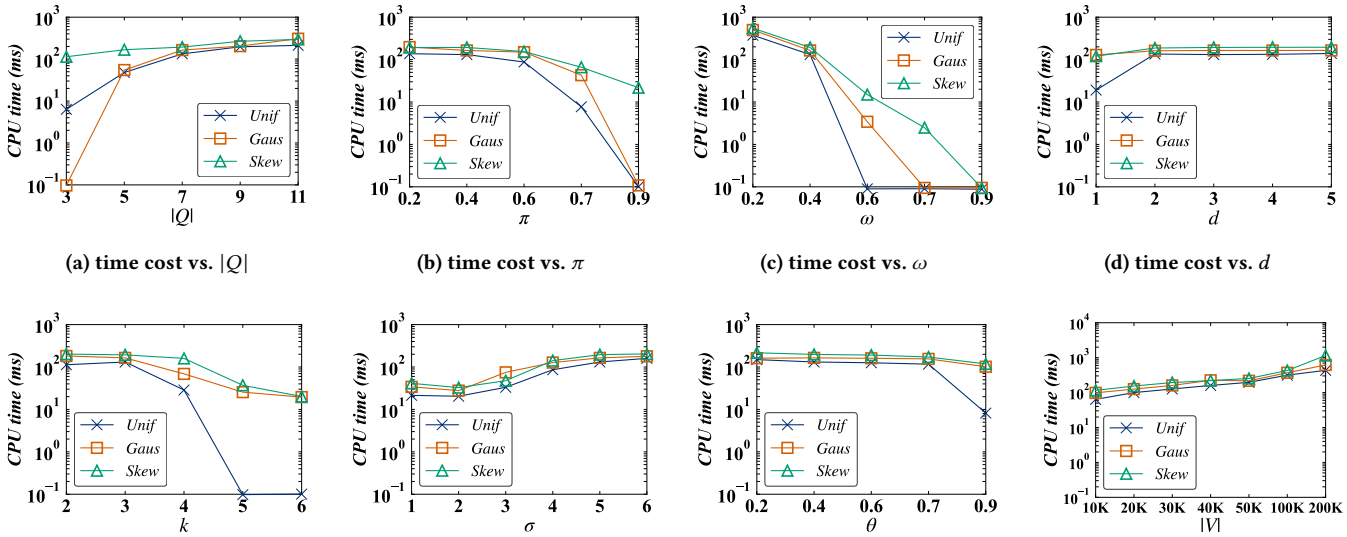
Performance vs. POIs Average Visiting Frequency Threshold π. Figure 5(b) analyzes $\pi \in 0.2, 0.4, 0.6, 0.7, 0.9$. As π increases, execution time decreases due to stronger pruning of POIs with low average visit frequency, which in turn reduces candidate users. Runtime drops from roughly 10^2 ms at $\pi = 0.2$ to near 10^0–10^1 ms at $\pi = 0.9$. This pruning operates after keyword-based filtering, where POIs that do not match the query keywords have already been removed. However, as π becomes larger, the constraint becomes much stricter, leading to a significant reduction in both POIs and their associated users, and thus a noticeable improvement in execution time.

Performance vs. Users Total Visiting Frequency Threshold ω. Figure 5(c) varies ω over $0.2, 0.4, 0.6, 0.7, 0.9$. Increasing ω significantly reduces candidate users, leading to notable runtime improvement. In fact, runtime decreases sharply between $\omega = 0.4$ and $\omega = 0.6$, and approaches minimal values for $\omega \geq 0.7$. In some distributions, the improvement spans nearly three orders of magnitude, indicating that ω-based pruning is one of the most dominant filtering mechanisms. Keyword distribution also affects pruning behavior, particularly in the $Unif$ and $Gaus$ datasets, where selective POI pruning further reduces computational cost.

Performance vs. Social Network Distance Threshold d. Figure 5(d) varies d from 1 to 5 hops. As expected, runtime increases with d because expanding the social radius enlarges the candidate subgraph and increases the number of users that must be examined. Execution time grows from approximately 10^1 ms to around 10^2 ms across the tested range. This behavior indicates that the evaluated datasets are relatively dense and well connected, as increasing d quickly incorporates additional users into the search space. Nevertheless, the growth remains smooth and bounded within a single order of magnitude, without exhibiting exponential escalation. This demonstrates that the pruning mechanisms effectively control search expansion and ensure good scalability with respect to the social distance threshold.

Performance vs. Triangle Support Threshold k. Figure 5(e) evaluates $k \in 2, 3, 4, 5, 6$. Execution time decreases as k increases, since stronger structural constraints enable more aggressive pruning of low-support users. For higher values of k (e.g., 5 or 6), runtime drops close to 10^0–10^1 ms in certain datasets. This indicates that a large portion of edges have support values below 5 or 6, and thus are eliminated early when stricter cohesiveness constraints are enforced. Consequently, higher k values significantly reduce candidate density and overall computation time.

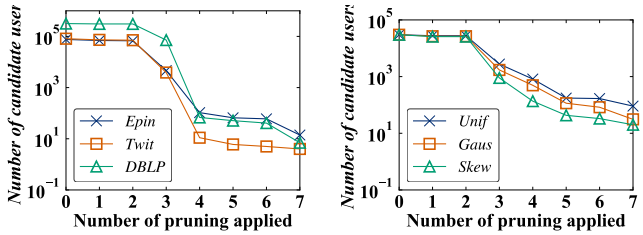
Performance vs. Road Network Distance Threshold σ. Figure 5(f) varies σ from 1 to 6. Larger σ values expand the spatial search region, increasing the number of candidate POIs and thus the runtime. Execution time increases steadily from approximately 10^1 ms to nearly 10^2 ms as σ grows, reflecting the expansion of the spatial search region. However, the growth remains smooth and bounded within a single order of magnitude, indicating stable spatial scalability and effective pruning control without noticeable



(a) time cost vs. $|Q|$ (b) time cost vs. π (c) time cost vs. ω (d) time cost vs. d

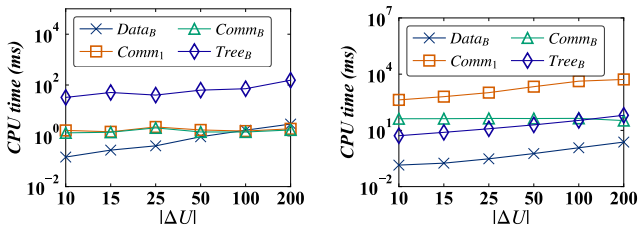
(e) time cost vs. k (f) time cost vs. σ (g) time cost vs. θ (h) time cost vs. N

Figure 5: Overall *KCS-BSSN* performance comparison for different parameter settings.



(a) real-world graphs (b) synthetic graphs

Figure 6: The ablation study of our *KCS-BSSN* pruning strategies on real/synthetic graphs, in terms of the pruning power.



(a) real-world graph ($Twit$) (b) synthetic graph ($Unif$)

Figure 7: The *KCS-TBSSN* performance of temporal dynamic updates (insertion) vs. real ($Twit$) and synthetic ($Unif$) graphs.

performance spikes.

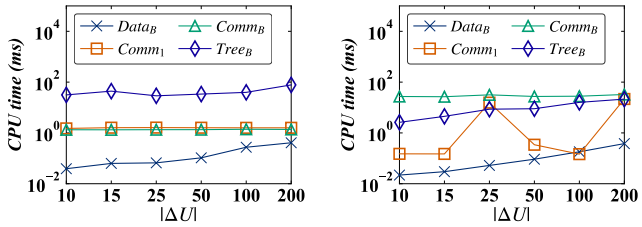
Performance vs. Influence Score Threshold θ. Figure 5(g) varies $\theta \in 0.2, 0.4, 0.6, 0.7, 0.9$. Since influence pruning is applied in a late filtering stage, increasing θ gradually reduces candidate users and slightly decreases execution time. The reduction is moderate compared to ω and k, confirming that influence acts mainly as a refinement constraint rather than a primary pruning driver.

Performance vs. Number of Users $|V|$. Figure 5(h) evaluates scalability for $|V|$ ranging from $10K$ to $200K$. Runtime increases with network size across all datasets ($Unif$, $Gaus$, $Skew$), as larger networks enlarge both social and spatial search spaces. The CPU time is about 10^2 ms for our default setting $|V| = 30K$, even under less restrictive parameter settings, the runtime remains within practical bounds. At $|V| = 200K$, execution time is around 10^3 ms for all three datasets, indicating that the method scales efficiently and maintains stable performance even for large-scale networks.

Pruning Power Analysis on Real/Synthetic Datasets. Figures 6 present an incremental evaluation of pruning effectiveness on real-world datasets ($Epin$, $Twit$, $DBLP$) and synthetic datasets ($Unif$, $Gaus$, $Skew$) under default settings. We measure the reduction in candidate users across eight stages, starting from the initial user set (Stage 0) and cumulatively activating: (1) Keyword-Based, (2) π-Based, (3) ω-Based, (4) Social-Distance-Based, (5) Structural Cohesiveness, (6) Spatial-Distance-Based, and (7) Influence-Based pruning. Across all datasets, the candidate set decreases steadily at each stage. A significant reduction is observed after ω-based pruning, highlighting its strong filtering power. In contrast, Keyword-Based and π-Based pruning show more moderate effects due to the high density of POIs associated with multiple keywords. Overall, the layered pruning strategy effectively reduces the search space and improves query efficiency in both real and synthetic environments.

7.3 *KCS-TBSSN* Performance Evaluation

Temporal Dynamic Updates / Insertion Performance Analysis on Real ($Twit$) and Synthetic ($Unif$) *TBSSN* Graphs. Figure 7 illustrates the *KCS-TBSSN* performance comparison of keyword-based community search over dynamic real-world ($Twit$) and synthetic ($Unif$) *TBSSN* graphs, where the batch size, $|\Delta U|$, of insertion updates varies from 10 to 200, with all other community



(a) real-world graph ($Twit$) **(b) synthetic graph ($Unif$)**

Figure 8: The *KCS-TBSSN* performance of temporal dynamic updates (deletion) vs. real ($Twit$) and synthetic ($Unif$) graphs.

parameters held at default settings. The results reveal distinct performance trends across the maintenance categories. In the real graph ($Twit$), as shown in Figure 7(a), processing times for all strategies remain relatively stable despite increases in batch size. Within this environment, $Tree_B$ consistently requires the highest CPU time, maintaining a range between 10 and 100 ms, while $Comm_1$ and $Comm_B$ demonstrate nearly identical efficient performance hovering near 1ms, and $Data_B$ functions as the most efficient strategy. In contrast, the synthetic graph ($Unif$) in Figure 7(b) exhibits a more pronounced upward trend in CPU time as batch sizes grow. Specifically, $Comm_1$ is significantly more expensive, rising from approximately 100 ms to nearly 10^4 ms at 200 users, largely because communities in this dataset are generally larger than those in the $Twit$ dataset. In both datasets, $Comm_1$ requires more time because many selected users not initially in a community must undergo the refinement process. However, our proposed $Comm_B$ remains highly stable and efficient, consistently performing near 10 ms, which validates that batch updates scales significantly better than individual updates under synthetic workloads for insertion operations.

Temporal Dynamic Updates / Deletion Performance Analysis on Real ($Twit$) and Synthetic ($Unif$) *TBSSN* Graphs. Figure 8 illustrates the *KCS-TBSSN* performance upon deletion operations, simulating check-in expiration within the sliding window model. In this experiment, the batch size $|\Delta U|$ also varies from 10 to 200, while all other community parameters are set to their default values. For the real-world graph ($Twit$), as shown in Figure 8(a), $Tree_B$ remains the most computationally intensive operation, with the CPU time stabilizing between 10 ms and 100 ms. Conversely, the community maintenance strategies $Comm_1$ and $Comm_B$ (with individual updates each time and batch updates, respectively) exhibit high ad similar efficiency (i.e. around 1ms), indicating that most users in $|\Delta U|$ do not belong to community answers (w.r.t. registered community search queries), allowing the update process to bypass them. $Data_B$ consistently emerges as the most efficient strategy, re-computing metrics with minimal time cost. For synthetic graph ($Unif$) in Figure 8(b), $Comm_1$ performs well across most trials but suffers significant latency spikes at $|\Delta U| = 25$ and 200, while $Comm_B$ maintains stable performance. These spikes suggest that specific batch sizes included more users belonging to community answers, triggering more intensive validation. While $Tree_B$ exhibits an upward trend in the CPU time as batch sizes increase, $Data_B$ remains remarkably stable and efficient. These experimental results confirm that the batch maintenance approach effectively reduces the computational overhead of expiring temporal data, except in cases where individual user validation ($Comm_1$) may be faster for users who are not part of existing community answers.

The experimental results for other real/synthetic graphs are similar and thus we do not report them here.

8 RELATED WORK

Community Search (CS). Community search strategies are generally categorized into four main methodologies [42]: peeling [22, 41, 43], expansion [22, 41], pruning [22, 43], and indexing-based strategies [23, 39]. The primary objective of CS is to retrieve a subgraph that satisfies specific constraints while containing a user-specified query vertex. Existing literature focuses on diverse constraints, including structural cohesiveness, keyword (attribute) homogeneity, embedded bipartite cores, community size, member influence, and spatial proximity.

CS in Heterogeneous Graphs. CS in heterogeneous graphs where different vertex and edge types coexist—aims to identify communities spanning multiple entity types. Chen et al. [6] proposed algorithms for both regular and large-scale heterogeneous graphs to identify single-type and multi-type communities. Zhou et al. [43] introduced the concept of heterogeneous influential communities based on meta-path core models [12, 27]. For a comprehensive overview of heterogeneous community models, refer to the recent survey by [42].

CS in Bipartite Graphs. Bipartite graphs are a specialized form of heterogeneous graphs consisting of two disjoint vertex sets. Research in this area often focuses on the core structure. For instance, the community with (ω, β)-core structure, where upper-layer vertices have a minimum degree of ω and lower-layer vertices have a minimum degree of β [9]. To manage community scale, size constraints are often imposed, such that the upper-layer size $\leq \mathfrak{x}$ and the lower-layer size $\leq \mathfrak{z}$ [41]. Zhou et al. [41] developed peeling and expansion algorithms for these structures, while Li et al. [22] investigated the Maximal Size Constraint CS (MSCC), proving it to be NP-Hard and proposing the Expand-and-Filter (EFA++) algorithm for acceleration. To incorporate social impact, Zhang et al. [39] addressed the (ω, β)-influential community problem, while Li et al. [23] extended CS to temporal bipartite graphs using time-windowed (ω, β)-cores. Furthermore, Xu et al. [35] explored attributed bipartite graphs to maximize attribute similarity, and Wang et al. [32] demonstrated that finding k-core communities with combined textual and numerical attributes is NP-Hard in in heterogeneous graph.

CS in Spatial-Social Networks. Recent research has increasingly focused on geo-social or spatial-social networks [3, 15–17, 29, 33]. Haldar et al. [16, 17] investigated location-based social networks, utilizing query locations to identify connected subgraphs that satisfy k-core requirements and distance thresholds. Similarly, Wang et al. [29] provided solutions for k-core structures with road-network radius constraints. Wu et al. [33] proposed top-n community retrieval based on proximity to a query user q, on the social network with k-core cohesive and radius constraint. Rai et al. [24] focused on retrieving communities similar to a given query community on the road network. Guo et al. [15] investigated multi-attributed CS in road-social networks, balancing structural

cohesiveness with road-network distance. Similarly, Ahmed et al. [2] incorporated both member influence and spatial proximity into multi-attributed frameworks.

Summary. The query proposed in this paper introduces the (ω, π)-*keyword-core*, which integrates a unique combination of constraints: keyword similarity, user influence, structural cohesion via (k, d)-*truss*, and road network distance $avg_dist_r(.)$. Due to this multi-faceted constraint set, existing techniques cannot be applied directly to the *KCS-BSSN* problem.

9 CONCLUSIONS

In this paper, we propose a novel community search query, called Keyword-based Community Search in Bipartite Spatial-Social Networks (*KCS-BSSN*), motivated by many real-world applications. The *KCS-BSSN* query identifies a cohesive community that satisfies the (ω, π)-*keyword-core* constraint, and ensures social influence, while considering both social connectivity and spatial proximity. To efficiently process the query, we develop effective pruning techniques to eliminate non-promising users early. We also design a novel indexing tree with a tailored cost model that integrates social and road network information. Based on this framework, we propose a two-phase algorithm that generates candidate communities and refines them to obtain exact results. Then, we introduced a temporal extension (*TBSSN*) to the *KCS-BSSN* model, integrating time-sensitive pruning to filter historical data. This approach ensures that discovered communities are socially and spatially cohesive while reflecting contemporary user behaviors. Extensive experiments on real and synthetic datasets demonstrate the efficiency and effectiveness of our proposed methods

REFERENCES

[1] A. Acquisti and R. Gross. Imagined communities: Awareness, information sharing, and privacy on the facebook. In *International workshop on privacy enhancing technologies*, pages 36–58. Springer, 2006.

[2] A. Al-Baghdadi and X. Lian. Topic-based community search over spatial-social networks. *Proc. VLDB Endow.*, 13(12):2104–2117, jul 2020.

[3] A. Al-Baghdadi, G. Sharma, and X. Lian. Efficient processing of group planning queries over spatial-social networks. *IEEE Transactions on Knowledge and Data Engineering*, 34(5):2135–2147, 2022.

[4] N. Barbieri, F. Bonchi, and G. Manco. Topic-aware social influence propagation models. *Knowledge and information systems*, 37(3):555–584, 2013.

[5] V. Batagelj and M. Zaversnik. An o(m) algorithm for cores decomposition of networks. *CoRR*, cs.DS/0310049, 2003.

[6] G. Chen, F. Guo, Y. Wang, Y. Liu, P. Yu, H. Shen, and X. Cheng. Fcs-hgnn: Flexible multi-type community search in heterogeneous information networks. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, CIKM '24, page 207–217, New York, NY, USA, 2024. Association for Computing Machinery.

[7] S. Chen, J. Fan, G. Li, J. Feng, K.-l. Tan, and J. Tang. Online topic-aware influence maximization. *Proceedings of the VLDB Endowment*, 8(6):666–677, 2015.

[8] J. Cohen. Trusses: Cohesive subgraphs for social network analysis. *National security agency technical report*, 16(3.1), 2008.

[9] D. Ding, H. Li, Z. Huang, and N. Mamoulis. Efficient fault-tolerant group recommendation using alpha-beta-core. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, CIKM '17, page 2047–2050, New York, NY, USA, 2017. Association for Computing Machinery.

[10] Y. Fang, X. Huang, L. Qin, Y. Zhang, W. Zhang, R. Cheng, and X. Lin. A survey of community search over big graphs. *The VLDB Journal*, 29(1):353–392, 2020.

[11] Y. Fang, Z. Wang, R. Cheng, H. Wang, and J. Hu. Effective and efficient community search over large directed graphs. *IEEE Transactions on Knowledge and Data Engineering*, 31(11):2093–2107, 2019.

[12] Y. Fang, Y. Yang, W. Zhang, X. Lin, and X. Cao. Effective and efficient community search over large heterogeneous information networks. *Proc. VLDB Endow.*, 13(6):854–867, Feb. 2020.

[13] K. R. Gabriel and R. R. Sokal. A New Statistical Approach to Geographic Variation Analysis. *Systematic Biology*, 18(3):259–278, 09 1969.

[14] A. Gibbons. *Algorithmic graph theory.* Cambridge university press, 1985.

[15] F. Guo, Y. Yuan, G. Wang, X. Zhao, and H. Sun. Multi-attributed Community Search in Road-social Networks . In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 109–120, Los Alamitos, CA, USA, Apr. 2021. IEEE Computer Society.

[16] N. A. H. Haldar, J. Li, N. Akhtar, Y. Jia, and A. Mian. Co-engaged location group search in location-based social networks. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):2910–2926, 2024.

[17] N. A. H. Haldar, J. Li, M. E. Ali, T. Cai, Y. Chen, T. Sellis, and M. Reynolds. Top-k socio-spatial co-engaged location selection for social users. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):5325–5340, 2023.

[18] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, SIGMOD '14, page 1311–1322, New York, NY, USA, 2014. Association for Computing Machinery.

[19] X. Huang and L. V. S. Lakshmanan. Attribute-driven community search. *Proc. VLDB Endow.*, 10(9):949–960, may 2017.

[20] J. Leskovec and J. Mcauley. Learning to discover social circles in ego networks. In F. Pereira, C. Burges, L. Bottou, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012.

[21] F. Li, D. Cheng, M. Hadjieleftheriou, G. Kollios, and S.-H. Teng. On trip planning queries in spatial databases. In C. Bauzer Medeiros, M. J. Egenhofer, and E. Bertino, editors, *Advances in Spatial and Temporal Databases*, pages 273–290, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[22] M. Li, R. Borovica-Gajic, F. M. Choudhury, N. Cui, and L. Ding. Maximal size constraint community search over bipartite graphs. *Knowledge-Based Systems*, 297:111961, 2024.

[23] S. Li, K. Wang, X. Lin, W. Zhang, Y. He, and L. Yuan. Querying historical cohesive subgraphs over temporal bipartite graphs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 2503–2516, 2024.

[24] N. Rai and X. Lian. Top-k community similarity search over large-scale road networks. *IEEE Transactions on Knowledge and Data Engineering*, 35(10):10710–10721, 2023.

[25] M. Richardson, R. Agrawal, and P. Domingos. Trust management for the semantic web. In D. Fensel, K. Sycara, and J. Mylopoulos, editors, *The Semantic Web - ISWC 2003*, pages 351–368, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[26] S. B. Seidman. Network structure and minimum degree. *Social Networks*, 5(3):269–287, 1983.

[27] Y. Sun, J. Han, X. Yan, P. S. Yu, and T. Wu. Pathsim: meta path-based top-k similarity search in heterogeneous information networks. *Proc. VLDB Endow.*, 4(11):992–1003, Aug. 2011.

[28] J. Wang, A. P. de Vries, and M. J. T. Reinders. Unifying user-based and item-based collaborative filtering approaches by similarity fusion. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '06, page 501–508, New York, NY, USA, 2006. Association for Computing Machinery.

[29] K. Wang, S. Wang, X. Cao, and L. Qin. Efficient radius-bounded community search in geo-social networks. *IEEE Transactions on Knowledge and Data Engineering*, 34(9):4186–4200, 2022.

[30] K. Wang, W. Zhang, X. Lin, Y. Zhang, L. Qin, and Y. Zhang. Efficient and effective community search on large-scale bipartite graphs. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 85–96, 2021.

[31] Y. Wang, J. Liu, X. Xu, X. Ke, T. Wu, and X. Gou. Efficient and effective academic expert finding on heterogeneous graphs through (k, p)-core based embedding. *ACM Trans. Knowl. Discov. Data*, 17(6), Mar. 2023.

[32] Y. Wang, S. Ye, X. Xu, Y. Geng, Z. Zhao, X. Ke, and T. Wu. Scalable Community Search with Accuracy Guarantee on Attributed Graphs . In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 2737–2750, Los Alamitos, CA, USA, May 2024. IEEE Computer Society.

[33] Z. Wu, J. Xu, H. Zhang, Q. Bao, Qingsun, and Changbengzhou. A progressive approach for neighboring geosocial communities search over large spatial graphs. *IEEE Access*, 10:57012–57024, 2022.

[34] H. Xie, Q. Liu, C. Luo, Y. Zhou, and Y. Gao. Truss-based why-not community search. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD '25, page 3309–3320, New York, NY, USA, 2025. Association for Computing Machinery.

[35] Z. Xu, Y. Zhang, L. Yuan, Y. Qian, Z. Chen, M. Zhou, Q. Mao, and W. Pan. Effective community search on large attributed bipartite graphs. *International Journal of Pattern Recognition and Artificial Intelligence*, 37(02):2359002, 2023.

[36] J. Yang and J. Leskovec. Defining and evaluating network communities based on ground-truth. In *Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics*, MDS '12, New York, NY, USA, 2012. Association for Computing Machinery.

[37] L. Yuan, L. Qin, X. Lin, L. Chang, and W. Zhang. I/o efficient ecc graph decomposition via graph reduction. *The VLDB Journal*, 26(2):275–300, Apr. 2017.

[38] L. Yuan, L. Qin, W. Zhang, L. Chang, and J. Yang. Index-based densest clique percolation community search in networks. *IEEE Transactions on Knowledge and*

Data Engineering, 30(5):922–935, 2018.
[39] Y. Zhang, Z. Hua, L. Yuan, and Z. Chen. Top-r influential community search in bipartite graphs. In *VLDB 2025 Workshop on Large Scale Graph Data Analytics (LSGDA)*, 2025.
[40] Y. Zhang and J. X. Yu. Unboundedness and efficiency of truss maintenance in evolving graphs. In *Proceedings of the 2019 International Conference on Management of Data*, SIGMOD '19, page 1024–1041, New York, NY, USA, 2019. Association for Computing Machinery.
[41] K. Zhou, J. Xin, J. Chen, X. Zhang, B. Wang, and Z. Wang. Effective and efficient community search with size constraint on bipartite graphs. *Inf. Sci.*, 647(C), Nov. 2023.
[42] L. Zhou, J. Wang, Y. Song, L. Wang, and H. Chen. Community search over heterogeneous information networks: A survey. *ACM Comput. Surv.*, 58(4), Oct. 2025.
[43] Y. Zhou, Y. Fang, W. Luo, and Y. Ye. Influential community search over large heterogeneous information networks. *Proc. VLDB Endow.*, 16(8):2047–2060, Apr. 2023.

A THE PROOFS

Proof of the Lemma 1

Proof. From our lemma assumption, if $p.K \cap Q = \emptyset$ holds for all POIs $p \in V_p$ that user u visited, then, for any vertex subset, V'_p, of V_p in a community (subgraph) of graph G_b, we have $p.K \cap Q = \emptyset$ for all POIs $p \in V'_p$ (as $V'_p \subseteq V_p$). Based on Definition 7, user u cannot be in the (ω, π)-*keyword-core*, and can thus be safely pruned. □

Proof of the Lemma 2

Proof. Since $ub_f_{sum}(u, V'_p)$ is an upper bound of $f_{sum}(u, V'_p)$, we have $ub_f_{sum}(u, V'_p) \geq f_{sum}(u, V'_p)$. From our lemma assumption, it holds that $ub_f_{sum}(u, V'_p) < \omega$. Hence, by the inequality transition, we can derive that $f_{sum}(u, V'_p) < \omega$, and safely prune user u with a low $f_{sum}(u, V'_p)$ value. □

Proof of the Lemma 3

Proof. Since $ub_f_{avg}(u)$ is an upper bound of $f_{u,p}$ for all p visited by u, we have $ub_f_{avg}(u) \geq f_{avg}(V'_s, p)$. From our lemma assumption, we have $ub_f_{avg}(u) < \pi$. Hence, by the inequality transition, we can derive $f_{avg}(V'_s, p) < \pi$, and safely prune the user u with a low $f_{avg}(V'_s, p)$ value. □

Proof of the Lemma 5

Proof. Since $ub_sup(u)$ is the upper bound of $sup(e)$ of all edges between user u and its neighbors, then, according to Definition 3, we can safely prune u if $ub_sup(u) < k - 2$. □

B THE INDEX PIVOTS SELECTION FOR LEAF NODES

Bipartite Structure. Since each $key_j \in u.K$, has the aggregate $key_j.f_{sum}$ and $key_j.f_{max}$ (as given in Eq. (10) and Eq. (11), respectively). To make these aggregations comparable, we normalize $key_j.f_{sum}$ and $key_j.f_{max}$ into the range [0-1] by dividing it by the largest f_{sum} and f_{max}, respectively. Then, for any two users $u, v \in G_s$, we calculate the bipartite structure score $bs_score(u, v)$ as follows:

$$bs_score(u,v) = \sum_{\forall key_j \in \{u.K \cap v.K\}} \frac{(u.key_j.f_{sum} + v.key_j.f_{sum})}{largest(f_{sum})} + \frac{(u.key_j.f_{max} + v.key_j.f_{max})}{largest(f_{max})} \tag{24}$$

, where $d = d_{max}$.

The higher the score $bs_score(u, v)$, the higher the probability of sharing similar keywords, which implies that their interests are more similar.

Spatial Structure. We calculate the spatial structure score based on the average distance we provide in Eq. (3). Then, for any users $u, v \in G_s$. we calculate $rs_score(u, v)$ as follows:

$$rs_score(u,v) = \frac{\frac{\sum_{\forall p \in v.L} avg_dist_r(u,p)}{|v.L|}}{largest(rs_score(.))} \tag{25}$$

The lower the value of $rs_score(u, v)$ between user u and v, the closer the distance between them. Similarly to $bs_score(.)$, we normalize $rs_score(.)$ to the range [0-1] by dividing it to the largest $rs_score(.)$.

Social Structure. We must consider (k, d)-*truss* and topic influences among users to measure social closeness in the social network. Furthermore, we need to consider social distance and social cohesiveness when calculating (k, d)-*truss*, as stated in Definition 3. Thus, for any two users $u, v \in G_s$, we calculate the social cohesion by finding $sum_sup(u, v) = ub_sup(u) + ub_sup(v)$, where $ub_sup()$ is given in Eq. (6). On the other hand, we use Eq. (4) to calculate the influence $ub_ISF(u, v)$. In both $sum_sup(.,.)$ and Eq. (4), values generally lead to greater cohesiveness. In contrast, Definition 3 requires a small social distance $dist_s(u, v)$ between users. To make all $sum_sup(.), ub_ISF(u, v)$, and $dist_s(u, v)$ comparable, we normalize all to the range [0-1] by dividing it by the largest $sum_sup(.)$, $ub_ISF(.)$, $dist_s(.)$, respectively. Finally, to determine the social structure score, we use the following calculation:

$$\begin{aligned} & ss_score(u,v) \\ = & \frac{sum_sup(u,v)}{largest(sum_sup(.))} + \frac{ub_ISF(u,v)}{largest(ub_ISF(.))} + \frac{(1 - dist_s(u,v))}{largest(dist_s(.))} \end{aligned} \tag{26}$$

Obviously, the higher the $ss_score(u, v)$ score indicates more coherence, a closer social distance, and a higher degree of influence between u and v.

C THE INDEX PIVOTS SELECTION FOR NON-LEAF NODES

Bipartite Structure Score for Non-Leaf Nodes. As explained in Section 4.1, each node has a set $N.K$, where each $key_j \in N.K$ is associated with $key_j.ub_f_{sum}$ and $key_j.ub_f_{max}$ (as given in Eq. (12) and Eq. (13), respectively). To make $key_j.ub_f_{sum}$ and $key_j.ub_f_{max}$ comparable, we normalize the values to the range [0-1] by dividing them to the largest $bs_f_{sum}(.)$ and $bs_f_{max}(.)$, respectively. Then, for a user index pivot $piv'_i \in \mathbb{P}'_{index}$ ($\mathbb{P}'_{index} \subset \mathbb{P}_{index}$), we calculate the bipartite structure score for a node N as follows:

$$\begin{aligned} & bs_score_node(N, piv'_i) \\ = \; & bs_ub_f_{sum}(N, piv'_i) + bs_ub_f_{max}(N, piv'_i), \end{aligned} \tag{27}$$

where $bs_ub_f_{sum}(N, piv'_i)$ and $bs_ub_f_{max}(N, piv'_i)$ calculated as follows:

$$bs_ub_f_{sum}(N, piv'_i) = \frac{\sum_{\forall key_j \in \{N.K \cap piv'_i.K\}} N.key_j.ub_f_{sum} + piv'_i.key_j.ub_f_{sum}}{largest(bs_f_{sum}(.))} \quad (28)$$

$$bs_ub_f_{max}(N, piv'_i) = \frac{\sum_{\forall key_j \in \{N.K \cap piv'_i.K\}} N.key_j.ub_f_{max} + piv'_i.key_j.ub_f_{max}}{largest(bs_f_{max}(.))}$$

where $d = d_{max}$.

The higher $bs_score_node(N, piv'_i)$ score implies that the interest in visiting similar places is more similar between the users in node N and the pivot user piv'_i.

Social Structure Score for Non-Leaf Nodes. The social structure score is mainly dependent on constraints within the social network such as the (k, d)-$truss$ and topic influences among users. We use the parameters of node N to calculate the social structure score between N and a $piv'_i \in \mathbb{P}'_{index}$. We calculate the social structure score for a node N and a $piv'_i \in \mathbb{P}'_{index}$ as follows:

$$\begin{aligned} & ss_score_node(N, piv'_i) \quad (29) \\ = \; & sum_sup_node(N, piv'_i) + max_ub_ISF(N, piv'_i) \\ & +(1 - (lb_dist_s(N, piv'_i)) \end{aligned}$$

, where the normalized $sum_sup_node(N, piv'_i)$ calculated as follows:

$$sum_sup_node(N, piv'_i) = \frac{ub_sup(N) + ub_sup(piv'_i)}{largest(sum_sup_node(.))}$$

, where $ub_sup(N)$ and $ub_sup(piv'_i)$ are given in Eqs. (14) and (6), respectively.

In addition, we calculate the normalized $max_ub_ISF(N, piv'_i)$ as follows:

$$max_ub_ISF(N, piv'_i) = \frac{ub_ISF(piv'_i, N)}{largest(max_ub_ISF(.))}$$

where $ub_ISF(piv'_i, N)$ is given in Eqs. (18).

Finally, we compute the distance lower bound $lb_dist_s(piv'_i, N)$, where $lb_dist_s(.)$ is given in Eq. (20).We normalize $lb_dist_s(.)$ to the range [0-1].

Based on Definition 9, we want to assign nodes to subgroups that maximize both $sum_sup_node(.)$ and $max_ub_ISF(.)$ and minimize $lb_dist_s(.)$, which generally leads to greater cohesiveness and a smaller social distance.